

آموزش صفر تا صد – همراه با جواب‌های کامل

مصاحبه شغلی فرانت‌اند از ترس تا تسلط

React و ۸۹ – Next.js سوال با جواب، ۸ تمرین لایوکدینگ و برنامه
شکستن ترس

Live Coding

۸۹ سوال با جواب

رزومه و پورتفولیو

شکستن ترس

مذاکره حقوق

System Design

۱۵ فصل · جواب نمونه STAR · چک‌لیست روز مصاحبه · برنامه ۹۰ روز اول

فهرست مطالب

فلسفه دوره و برنامه ۳ هفته	شروع دوره	1
۳ ریشه ترس، reframe، thinking out loud	فصل ۱ – روانشناسی ترس مصاحبه	2
رزومه عددی، ۳ پروژه پرچمدار، LinkedIn	فصل ۲ – رزومه و پورتفولیو	3
قیف اپلای، DM، جدول پیگیری	فصل ۳ – جستجو و اپلای	4
screening تا offer، جدول آمادگی	فصل ۴ – مراحل مصاحبه	5
event loop، closure، debounce	فصل ۵ – ۱۸ سوال JS/TS	6
VDOM، keys، memo، React 19	فصل ۶ – ۲۰ سوال React	7
SSG/ISR، RSC، Server Action	فصل ۷ – ۱۶ سوال Next.js	8
۸ تمرین با کد کامل و پروتکل رفتار	فصل ۸ – ۸ تمرین Live Coding	9
قالب ۵ مرحله + ۲ پاسخ نمونه	فصل ۹ – System Design	10
۴ STAR جواب نمونه کامل	فصل ۱۰ – مصاحبه رفتاری (STAR)	11
سوالات تو، red flags، مذاکره	فصل ۱۱ – سوالات تو و حقوق	12
follow-up، رد شدن، ۹۰ روز اول	فصل ۱۲ – بعد از مصاحبه	13
چک‌لیست‌ها + گلاساری + نقشه راه	فصل ۱۳ – چیت‌شیت و گلاساری	14
HTML/CSS از هندبوک فرانت – بدون تکرار	فصل ۱۴ – ۲۰ سوال HTML/CSS	15
DOM/مرورگر + خروجی‌بگوهای کوپیز JS	فصل ۱۵ – مرورگر، DOM و تله‌های JS	16

دوره جامع مصاحبه شغلی فرانت‌اند – با جواب‌های کامل 🎓

موضوع: مصاحبه استخدامی برای برنامه‌نویسان – مخصوص فرانت‌کارهای React / Next.js مخاطب: از اولین مصاحبه عمرت تا جویای کاری که مدت‌هاست مصاحبه می‌دهد و هنوز استرس دارد هدف: حذف ترس، استرس و «فوبیای» مصاحبه – با آمادگی واقعی: هر سوال با جواب نمونه

🎯 فلسفه این دوره: مصاحبه یک «گفتگوی دوطرفه» است، نه بازخواست

ترس مصاحبه از سه باور غلط می‌آید – و هر سه را در این دوره می‌شکنیم:

باور غلط 🤪	حقیقت ✅
«باید همه‌چیز را بدانم»	مصاحبه‌گر هم می‌داند تو همه‌چیز را نمی‌دانی؛ نحوه‌ی برخوردت با ندانستن را می‌سنجد
«یک اشتباه = رد شدن»	مصاحبه مجموعه‌ای از سیگنال‌هاست؛ یک اشتباه جبران‌پذیر است
«مصاحبه‌گر دشمن من است»	او هم می‌خواهد تو رد شوی نه – او به کمک نیاز دارد و امیدوار است تو همان باشی

و مهم‌ترین بازچینش ذهنی: تو هم مصاحبه می‌کنی. شرکت را می‌سنجی: آیا رشد می‌دهم؟ آیا تیمش سالم است؟ وقتی این را بپذیری، پوزیشن قدرت عوض می‌شود و نیمی از ترس می‌میرد.

📖 نقشه دوره

بخش اول – آمادگی (قبل از مصاحبه)

فصل	موضوع
فصل ۱	روانشناسی ترس مصاحبه و شکستن آن
فصل ۲	رزومه، پورتفولیو و گیت‌هاب آماده مصاحبه
فصل ۳	جستجو، اپلای و پیگیری
فصل ۴	مراحل مصاحبه: چه چیزی در هر مرحله می‌گذرد

بخش دوم – سوالات فنی با جواب کامل

فصل	موضوع
فصل ۵	۱۸ سوال JavaScript/TypeScript + جواب
فصل ۶	۲۰ سوال React + جواب
فصل ۷	۱۶ سوال Next.js + جواب
فصل ۸	۸ تمرین لایو-کدینگ رایج + کد کامل جواب

بخش سوم – مهارت‌های غیرفنی

فصل	موضوع
فصل ۹	طراحی سیستم برای فرانت‌کار + پاسخ نمونه
فصل ۱۰	مصاحبه رفتاری با STAR + جواب‌های نمونه
فصل ۱۱	سوالاتی که تو می‌پرسی + Red Flags + حقوق
فصل ۱۲	بعد از مصاحبه: follow-up، رد شدن، مذاکره، ۹۰ روز اول
فصل ۱۳	چیت‌شیت + چک‌لیست‌ها + گلاساری + نقشه راه

بخش چهارم – سوالات فنی تکمیلی (از رپوهای مرجع گیت‌هاب)

فصل	موضوع
فصل ۱۴	۲۰ سوال HTML/CSS + جواب – استخراج‌شده از Front End Interview Handbook
فصل ۱۵	۱۵ سوال مرورگر، DOM و تله‌های JS – استخراج‌شده از هندبوک فرانت + کوئیز JavaScript Questions

برنامه تمرین پیشنهادی (۳ هفته)

هفته	کار
هفته ۱	فصل ۱-۴ + رزومه‌ات را بازنویسی کن + فصل ۵ را بخوان و جواب‌ها را بلند بگو
هفته ۲	فصل ۶-۸ + هر روز ۲ تمرین لایو-کدینگ با تایمر
هفته ۳	فصل ۹-۱۲ + ۲ مصاحبه شبیه‌سازی (از دوستت بخواه سوال بپرسد!) + چیت‌شیت فصل ۱۳ + فصل‌های ۱۴-۱۵ تکمیلی

قانون طلایی: جواب سوالات را **بلند بگو**، نه فقط بخوان. مغز تو در مصاحبه با «گفتن» روبرو می‌شود، نه با «دانستن».

مصاحبه‌ی شبیه‌سازی‌شده، واکسن ترس است.

نسخه PDF 

نسخه PDF رنگی راست‌به‌چپ (فونت وزیرمتن) در همین پوشه: **«دوره کامل مصاحبه شغلی فرانت‌اند.pdf»**. بازسازی:

رندر Chrome headless + `python build/build_pdf.py`

فصل ۱ – روانشناسی ترس مصاحبه: دشمن واقعی «ندانستن چه رخ می‌دهد» است

🎯 هدف این فصل: ترس مصاحبه یک مهارت‌زدایی‌شدنی است، نه یک ویژگی شخصیتی. سه منبع ترس را ریشه‌یابی می‌کنیم و ابزار عملی هر کدام را می‌گیریم.

۱.۱ – ترس تو از کجا می‌آید؟ (سه ریشه)

ریشه ترس	صدای داخل سرت	پادزهر این دوره
ترس از ناشناخته	«نمی‌دونم چی می‌پرسن!»	فصل ۴ (مراحل) + فصل ۵-۸ (سوالات با جواب) – ناشناخته را شناخته کن
ترس از قضاوت	«اگه بلد نباشم چیکار فکر می‌کنن؟»	فصل این – reframe و تکنیک «بلند فکر کردن»
ترس از پیامد	«اگه رد شم چی؟»	فصل ۱۲ – مصاحبه بازی اعداد است، نه داوری شخصیت

۱.۲ – Reframe: سه جایگزینی ذهنی ★

- ۱) «آزمون» → «گفتگوی کاری دوطرفه» تو فروشنده‌ی مهارت هستی و خریدار هم باید متقاعد شود. وقتی خریدار را هم دیدی، لحن تو از «بخشایش بخواه» به «حرفه‌ای با تجربه» تغییر می‌کند.
- ۲) «باید همه‌چیز را بدانم» → «بلدم فکر کنم، بلدم یاد بگیرم» هیچ دولوپری همه‌چیز را نمی‌داند – حتی مصاحبه‌گر. آنچه او واقعاً می‌خرد: توانایی حل مسئله + قابل‌همکاری بودن + یادگیری سریع. جواب «نمی‌دانم» درست، بهتر از جواب بلوف غلط است.
- ۳) «رد شدن = شکست من» → «رد شدن = عدم تطابق این موقعیت با من (یا برعکس)» مصاحبه‌ای که رد می‌شوی، هزینه‌ی صفر و تجربه‌ی مجانی دارد – و آمار می‌گوید هر مصاحبه، شانس بعدی را بالا می‌برد. فرمول واقعی: رد شدن‌ها جزء ضروری فرایندند، نه استثنا.

۱.۳ – Imposter Syndrome: «من واقعاً بلد نیستم، فقط شانس آوردم»

بیماری شماره یک دولوپرها (مخصوصاً خوب‌ها). علائم: «بقیه خیلی بهترند»، «هر لحظه لو می‌روم»، «مصاحبه‌گر حتماً بالاتر از من است».

- دسترسی دستاوردها:** ۲۰ خط بنویس «چه چیزهایی ساختم که کار می‌کند» (۶۰ پروژه‌ی گیت‌هابت!). وقتی مغزت گفت «تو بلد نیستی»، حرفش را با فکت پاس بده
- مقایسه با خودِ دیروز، نه با senior های لینکدین** – highlight های لینکدین، روزمره‌ی دیگران نیست
- دانشات را بیرون بریز:** آموزش دادن (بلاگ، به دوستت) بهترین اثبات بلد بودن است – و اتفاقاً همین دوره را داری می‌خوانی

۱.۴ – تکنیک‌های آرام‌سازی روز مصاحبه (عملی، نه شعار)

تکنیک	چطور	کی
جعبه گرم کردن (warm-up)	۲۰ دقیقه قبل، یک کامپوننت ساده بنویس یا داکيومنت بخوان	مغز کدنویس گرم می‌شود
تنفس ۴-۷-۸	۴ ثانیه دم، ۷ نگاه‌دار، ۸ بازدم – سه بار	۵ دقیقه قبل ورود/شروع کال
جمله‌ی لنگر	یک جمله آماده: «من ۶۰ پروژه واقعی ساختم؛ هر چه نپرسید، می‌توانم فکر کنم و یاد بگیرم»	وقتی ذهن خالی شد، آن را بلند در ذهنت بگو
آماده‌سازی محیط (آنلاین)	کامپیوتر، میکروفن، اینترنت، آدرس کال را یک ساعت قبل چک کن؛ لیوان آب کنار دست	۱ ساعت قبل
چیت‌شیت پیش‌مصاحبه	نیم‌صفحه: اسم مصاحبه‌گر، سه پروژه‌ی خودت، ۳ سوال برای پرسیدن	۱۰ دقیقه قبل نگاه
خواب و غذا	شب قبل نه مطالعه‌ی سنگین؛ روز قبل صبحانه	همیشه

۱.۵ – «بلند فکر کردن» (Thinking Out Loud): سوپر قدرت مصاحبه فنی

بزرگ‌ترین اشتباه کاندیداها: سکوت و فکر کردن. مصاحبه‌گر در سکوتِ تو هیچ سیگنالی نمی‌گیرد و فرض می‌کند گیر کرده‌ای.

قالب طلایی پاسخ به هر سوال فنی:

- «بزنم؟ cursor سوال را بازگو کن:» «یعنی برای لیست بلند، صفحه‌بندی ۱.»
- «بگذار مقایسه کنم cursor و offset – مسیر فکر را بلند بگو:» «دو راه دارم ۲.»
- «پایدارتره cursor ساده‌تره ولی در داده بزرگ کند و جابه‌جاست؛ offset» : بگو trade-off ۳.
- «انتخاب می‌کنم چون cursor، feed تصمیم + دلیل:» «برای ۴.»
- «رو. بلد نیستم، ولی رویکردم این می‌بود X ندانستن را شفاف بگو:» «جزئیات ۵.»

این پنج قدم حتی وقتی جواب نهایی اشتباه از آب دربیاید، سیگنال «دولوپر قابل» می‌فرستد — چون در کار واقعی هم دقیقاً همین‌طور فکر می‌کنی و با تیم حرف می‌زنی.

جمع‌بندی فصل

- سه ریشه ترس: ناشناخته / قضاوت / پیامد — هرکدام پادزهر مشخص دارد
- مصاحبه = گفتگوی دوطرفه؛ تو هم می‌سنجی
- «نمی‌دانم» حرفه‌ای بگو: بگو چه می‌دانی و چطور می‌فهمی
- Imposter syndrome: دفترچه دستاورد + مقایسه با خودِ دیروز
- روز مصاحبه: warm-up، تنفس، جمله لنگر، چیت‌شیت
- **Thinking Out Loud** = پنج قدم؛ مهم‌ترین مهارت این کل دوره

تمرین فصل ۱

1. سه ترسی که از مصاحبه داری را بنویس و برای هرکدام از جدول ۱.۱ پادزهرش را مشخص کن.
2. دفترچه دستاوردهای خودت را باز کن: ۱۰ آیتم بنویس (پروژه‌های گیت‌هابت، مسائلی که حل کردی...).
3. «نمی‌دانم» حرفه‌ای بنویس: مصاحبه‌گر می‌پرسد «WebSocket چطور کار می‌کند؟» و تو کار نکرده‌ای — جواب ۳ خطی بده (چه می‌دانی + چطور می‌فهمی).
4. جمله لنگر خودت را بنویس و سه بار بلند بخوان.
5. تمرین Thinking Out Loud: سوال «تفاوت useRef و useRef چیست؟» را با قالب ۵ قدم فصل، **بلند** جواب بده (جلوی آینه یا ضبط صدا!).

▼ جواب نمونه تمرین ۳ و ۵

تمرین ۳: «WebSocket» را در production استفاده نکرده‌ام، ولی می‌دانم یک کانال دوسویه‌ی ماندگار روی TCP است — برخلاف polling که مدام می‌پرسیم، سرور می‌تواند push کند. اگر پروژه بخواهد، اول داکيومنت و یک POC ساده روی چت می‌سازم — الگویی شبیه event listener هایی است که بلدم.

تمرین ۵ (قالب ۵ قدم): ۱. «یعنی فرق state با ref را پرسیدی؟» — ۲. «دو ابزار ذخیره بین رندها...» — ۳. «state تغییرش رندر می‌سازد؛ ref تغییرش رندر نمی‌سازد و برای DOM و مقادیر فنی است» — ۴. «پس مقدار نمایشی → state؛ تایمر/۵ — ref → DOM» (نمی‌دانی؟ بگو چه بخشی را دقیق‌تر می‌خواهی چک کنی!)

→ **فصل بعد:** رزومه و پورتفولیو — برگ اول معرفی تو.

فصل ۲ – رزومه، پورتفولیو و گیت‌هاب آماده مصاحبه

🎯 هدف: سه برگ معرفی تو – رزومه (۱۰ ثانیه‌ای دیده می‌شود)، پورتفولیو (۳ دقیقه‌ای بررسی می‌شود)، گیت‌هاب (۱۰ دقیقه‌ای). هر کدام را بهینه می‌کنیم.

۲.۱ – رزومه فرانت‌کار: قوانین بقا در ۱۰ ثانیه

recruiter هر رزومه را ۱۰~ ثانیه اسکن می‌کند. قوانین:

۱. یک صفحه (تا ۵ سال سابقه). دو صفحه فقط با سابقه جدی
۲. بالای صفحه: اسم + عنوان (Frontend Developer – React/Next.js) + لینک‌ها (گیت‌هاب، لینکدین، پورتفولیو) + راه ارتباطی
۳. خلاصه ۲-۳ خطی: کی هستی + بلدترین‌ها + یک عدد واقعی
۴. پروژه‌ها با نتیجه‌ی قابل‌سنجش – نه لیست وظایف
۵. مهارت‌ها: فقط چیزهایی که در مصاحبه ازشان دفاع می‌کنی

❌ به ✅: بازنویسی بولت‌های رزومه

- ❌ «React کار روی سایت شرکت با»
✅ «Server Actions و Next.js با checkout بازطراحی صفحه»
«(Lighthouse ۹۶) زمان لود از ۴ ثانیه به ۱.۲ ثانیه»
- ❌ «TypeScript آشنایی با»
✅ «runtime؛ حذف ۸۰٪ باگ‌های TypeScript مهاجرت پروژه ۴۰ هزارخطی به»
- ❌ «تیم‌ورک و مسئولیت‌پذیری»
✅ «؛ مربیگری دو جونیور PR + هفتگی ۳ Code review»

الگو: فعل قوی + فناوری + نتیجه/عدد. اعداد حتی تقریبی، رزومه را از «همه» جدا می‌کند.

[نام و نام خانوادگی] – Frontend Developer (React / Next.js)
📍 شهر | ✉ email | 🌐 github.com/you | 🌐 linkedin.com/in/you | 🌐 portfolio

TypeScript و Next.js (App Router): خلاصه: فرانت‌اند دولوپر با تمرکز بر UX. پروژه‌ی واقعی از صفر تا دپلوی؛ علاقه‌مند به پرفورمنس و [N] ساخت‌های

پروژه‌های منتخب:

- (دپلوی‌شده: لینک) Next.js, Prisma, PostgreSQL – [اسم پروژه]
- [چه ساختی، با چه چالشی، با چه نتیجه‌ای – یک خط]
- ...

مهارت‌ها: TypeScript, React, Next.js (App Router), Tailwind, Git, REST/GraphQL, SQL, Docker (پایه), Testing (پایه)

...:تحصیلات / دوره‌ها

⚠ سه اشتباه کشنده: رزومه Word با طراحی شلوغ (PDF بده!)، «مسلط به» برای ۲۰ تکنولوژی (سطح‌بندی کن: متخصص/کاربردی/آشنا)، عکس و اطلاعات خانوادگی (فقط مهارت).

۲.۲ – پورتفولیو: ۳ پروژه بهتر از ۳۰ تمرین

گیت‌هابت را می‌شکافند. چیزی که می‌بینند:

سه پروژه‌ی پرچمدار (قانون):

1. یک اپ کامل فول‌استک – Next.js + دیتابیس + Auth + دپلوی آنلاین (دوره‌های ۱، ۴، ۵ تو دقیقاً همین را می‌سازند!)
 2. یک پروژه با پرفورمنس/جزئیات Lighthouse – UX بالا، responsive، dark mode، skeleton
 3. یک پکیج/ابزار کوچک یا contribution به open source – نشان می‌دهد عمق داری
- هر پروژه باید داشته باشد: **README خوب** (چی/چرا/چطور اجرا/اسکرین‌شات)، دمو آنلاین، کد تمیز با کامیت‌های معنادار، و `env.example`.

🔑 ترفند: به هر پروژه‌ی پرچمدار، بخش «چالش‌ها و تصمیم‌ها» اضافه کن (مثلاً «چرا cursor pagination به جای offset؟»). مصاحبه‌گر همین‌ها را می‌پرسد – و تو جواب آماده داری.

۲.۳ – گیت‌هاب: پروفایل حرفه‌ای در ۱۵ دقیقه

- [] **Profile README** (ریپوی هم‌نام خودت): معرفی + تکنولوژی‌ها + لینک پورتفولیو
- [] پین کردن ۶ ریپوی برتر (فقط پرچم‌دارها!)
- [] هر ریپوی پرچمدار: README + دمو + کامیت تمیز (فصل ۳ دوره گیت)
- [] Contribution graph: منظم بودن مهم‌تر از تعداد (کمی از قبل، کمکی حالا نمی‌کند)

- [] حذف/آرشیو ریپوهای تمرینی بی-README – یا به یک ریپوی «practice-grounds» منتقل کن

۲.۴ – LinkedIn: مغناطیس recruiter

- عنوان: `Frontend Developer | React • Next.js • TypeScript` (کلمات کلیدی = جستجوی recruiter)
- About: ۴-۵ خط روایت (چه می‌سازی، چه می‌خواهی) – بدون کلیشه
- Experience با همان بولت‌های عددی رزومه
- **Open to work** روشن (فقط برای recruiter ها هم می‌شود تنظیم کرد)
- هفته‌ای یک پست فنی (اسکرین‌شات پروژه + یک نکته) = دیده‌شدن

۲.۵ – Tailoring: یک رزومه برای همه = رد شدن برای همه

برای هر شرکت ۵ دقیقه: عنوان خلاصه را با واژه‌های آگهی هماهنگ کن (اگر گفت SSR و SEO، در خلاصه‌ات این دو کلمه بیاید)، ترتیب مهارت‌ها را عوض کن، پروژه‌ی مرتبط‌تر را اول بگذار. و در پوشش/ایمیل اپلای، دو خط درباره‌ی آن شرکت بنویس («از محصول‌تان X را دیده‌ام؛ تجربه‌ی من در Y دقیقاً به کارش می‌آید»).

جمع‌بندی فصل

- رزومه: یک صفحه، PDF، بولت‌های «فعل+فناوری+عدد»، فقط دفاع‌پذیرها
- پورتفولیو: ۳ پرچم‌دار با README و دمو آنلاین + بخش «چالش‌ها و تصمیم‌ها»
- گیت‌هاب: Profile README، پین برترها، آرشیو تمرین‌ها
- LinkedIn: کلمات کلیدی + Open to work + پست هفتگی
- Tailoring: ۵ دقیقه برای هر شرکت

تمرین فصل ۲

1. رزومه‌ات را با قالب این فصل بازنویسی کن – حداقل ۴ بولت با عدد.
2. سه پروژه پرچم‌دار انتخاب کن و برای هرکدام بخش «چالش‌ها و تصمیم‌ها» بنویس (۳ بولت).
3. Profile README گیت‌هاب را بساز/به‌روز کن و ۶ ریپو پین کن.
4. برای یک آگهی واقعی، رزومه‌ات را tailor کن و تفاوت‌ها را یادداشت کن.
5. از یک دوست بخواه رزومه‌ات را در ۱۰ ثانیه ببیند و بگوید چه فهمید – اگر «فرانت‌کار React» نگفت، دوباره کار کن.

▼ چک لیست تمرین ۲ (نمونه پر شده)

- «چرا cursor pagination؟» – لیست ۱۰۰۰+ آیتمی، offset در صفحه‌های دور کند و با insert جدید آیتم تکراری می‌داد؛ cursor پایدار ماند و کوئری با ایندکس $O(\log n)$ شد.
- «چرا Server Action به جای Route Handler؟» – فرم داخلی سایت بود؛ RPC خودکار + اعتبارسنجی مشترک با zod + بدون endpoint عمومی.
- «چرا dark mode با CSS variable و نه دو build؟» – یک DOM، سوییچ بدون reload، هماهنگی با system preference؛ trade-off: فایل CSS بزرگ‌تر.

➔ **فصل بعد:** کجا و چطور اپلای کنیم – و چطور پیگیری کنیم بدون مزاحمت.

فصل ۳ – جستجو، اپلای و پیگیری: قیف شانس را پهن کن

🎯 هدف: استخدام یک بازی قیف است: دیدن → اپلای → جواب → مصاحبه → آفر. یاد می‌گیری قیف را پهن و هوشمند مدیریت کنی – و با رد شدن‌ها کنار بیایی.

۳.۱ – قیف استخدام (واقعیت عددی)

۱-۳ آفر 🎯 → ۴-۵ مرحله فنی → ۱۰-۱۵ مصاحبه مرحله اول → ۲۰-۳۰ جواب اولیه → ۱۰۰ اپلای هدفمند

این اعداد برای کاندیدای خوب هم است! پس: رد شدن‌ها جزء فرمول‌اند. اگر ۵ اپلای دادی و جواب نیامد، مشکل تو نیست – قیف تان باریک است؛ پهنش کن.

۳.۲ – کجا بگردیم؟ (کانال‌ها به ترتیب بازده)

کانال	بازده	چطور
معرفی (Referral) ★★★★★	بالاترین نرخ موفقیت	از آشنایان/هم‌دوره‌ای‌ها بپرس؛ پست لینکدین «به دنبال فرصت فرانت‌اند هستم»
LinkedIn ★★	عالی	جستجوی شغلی + Open to Work + پیام مستقیم به hiring manager
سایت‌های آگهی داخلی ★★	حجم بالا	فیلتر React/Next؛ هر روز چک
سایت مستقیم شرکت‌ها	خوب	شرکت‌هایی که دوست‌شان داری، بخش careers را مانیتور کن
GitHub / open source	بلندمدت	contribution = آفر بدون اپلای
کامیونیتی‌ها (دیسکورد/تلگرام فنی)	متوسط	فعال بودن = فرصت‌ها خودشان می‌آیند

۳.۳ – پیام recruiter / hiring manager (قالب آماده)

وقتی مستقیم DM می‌دهی – کوتاه، شخصی، لینک‌دار:

سلام [اسم]

پروژه‌های [شرکت] رو دنبال می‌کردم - مخصوصاً [یک جزئیات واقعی TypeScript و Next.js از محصولش]. من فرانت‌اند دولوپر با تمرکز روی هستم؛ آخرین کارم [یک خط: چه ساختی + نتیجه]

پورتفولیو: [لینک] | گیت‌هاب: [لینک]

اگر فرصتی برای تیم‌تون هست، خوشحال می‌شم صحبت کنیم!
ممنون از وقت‌تون!

سه قانون: (۱) یک جزئیات واقعی از آن شرکت ذکر کن (کپی‌پیست لو می‌رود)، (۲) لینک پورتفولیو حتماً، (۳) کوتاه - زیر ۶ خط.

۳.۴ - مدیریت اپلای‌ها: جدول پیگیری (Spreadsheet)!

شرکت	تاریخ اپلای	کانال	مرحله	پیگیری بعدی	یادداشت
شرکت A	1 خرداد	LinkedIn	مصاحبه فنی	-	React + NestJS می‌خواهند
شرکت B	2 خرداد	سایت	جواب نداده	8 خرداد follow-up	remote-only

چرا مهم است؟ (۱) یادت نمی‌ماند کی و کجا، (۲) **follow-up** قانون‌دار می‌شود: ۵-۷ روز بی‌جواب = یک ایمیل پیگیری کوتاه مودبانه (اغلب اپلیکیشن‌ها گم می‌شوند، نه رد)، (۳) الگوهای رد شدن را می‌بینی.

💡 **قانون ۲۴ ساعت:** هر مصاحبه‌ای که رفتی، همان روز در جدول بنویس چه پرسیدند - سوال‌ها تجمع می‌شوند و مصاحبه‌های بعدی‌ات را می‌سازند (دیتابیس شخصی سوالات!).

۳.۵ - Cover letter / پیام اپلای: کوتاه و شخصی

بیشتر جاها cover letter کامل نمی‌خواهند - یک پاراگراف هوشمند کافی است:

سلام، برای موقعیت فرانت‌اند درخواست می‌دهم.

کار می‌کنم و تازه‌ترین پروژه‌ام Next.js و React سه سال است با و درگاه پرداخت است SSR یک فروشگاه کامل با (دیپلوی‌شده: [لینک])
در آگهی به پرفورمنس اشاره کرده بودید - همین موضوع تمرکز اصلی (روی هر سه پروژه‌ی اخیرم ۹۵+ Lighthouse) من بوده.

رزومه: [لینک] - خوشحال می‌شوم صحبت کنیم.

الگو: موقعیت را نام ببر → یک تطابق مشخص → یک شاهد عددی → دعوت به گفتگو. چهار جمله. تمام.

جمع‌بندی فصل ✓

- کیف واقعی: ۱۰۰ اپلای ← ۳-۱ آفر؛ رد شدن‌ها فرمول‌اند
- کانال‌ها به ترتیب بازده: LinkedIn ← Referral ← سایت‌ها ← مستقیم ← کامیونیتی
- پیام مستقیم: زیر ۶ خط + جزئیات واقعی شرکت + لینک‌ها
- جدول پیگیری + قانون ۲۴ ساعت (ثبت سوال‌ها!) + follow-up بعد ۵-۷ روز
- ۴ = Cover letter جمله: موقعیت + تطابق + عدد + دعوت

تمرین فصل ۳ 📝

1. جدول پیگیری اپلای را در گوگل‌شیت بساز با ستون‌های فصل – از امروز پرش کن.
2. پیام DM به دو شرکت واقعی (که واقعاً می‌خواهی) بفرست – هر کدام با یک جزئیات متفاوت.
3. یک پاراگراف cover letter برای آگهی مورد علاقه‌ات بنویس – قالب ۴ جمله.
4. ۵ شرکت که محصولاتش را دوست داری لیست کن و صفحات careersشان را bookmark کن.
5. (روانی) یک پست لینکدین «به دنبال فرصت» بنویس – با لینک پورتفولیو.

▼ نمونه تمرین ۴-۵

شرکت‌ها: مثلاً دو فروشگاه آنلاین که UX خوبی دارند + یک سرویس SaaS داخلی + یک استودیو محصول + یک شرکت بلیط/سفر. چرا؟ چون «دوست داشتن محصول» در مصاحبه به پاسخ‌های بهتر تبدیل می‌شود و مصاحبه‌گر حس می‌کند تو را برای خودشان می‌خواهد.

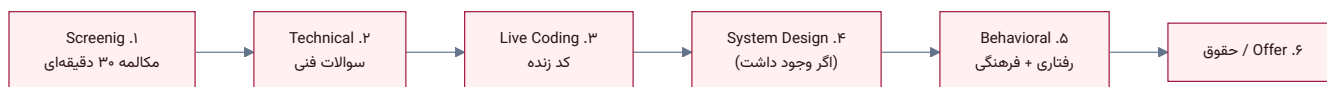
پست لینکدین نمونه: «چند ماهی است روی Next.js App Router تمرکز کردم – نتیجه‌اش این فروشگاه دموست که SSR، Server Actions و درگاه پرداخت دارد [لینک]. دنبال تیمی هستم که فرانت‌اند را جدی بگیرد. اگر جای مناسب می‌شناسید، خوشحال می‌شوم DM بدهید!» + اسکرین‌شات.

➔ **فصل بعد:** نقشه مراحل مصاحبه – دقیقاً چه چیزی در هر مرحله رخ می‌دهد.

فصل ۴ – مراحل مصاحبه: نقشه کامل میدان نبرد

🎯 هدف: بزرگترین منبع اضطراب، «ندانستن چه رخ می‌دهد» است. در این فصل کل خط لوله استخدام را قدم به قدم باز می‌کنیم – برای هر مرحله: چه می‌گذرد، چه می‌خواهند، چطور آماده شوی.

۴.۱ – نقشه کامل (مراحل معمول در شرکت‌ها)



⚠️ همه شرکت‌ها همه مراحل را ندارند – بسیاری از شرکت‌های داخلی: یک مکالمه فنی + یک جلسه با مدیر. برخی بین‌المللی‌ها: ۴-۵ مرحله در چند هفته. همیشه در اولین کال بپرس: «فرایند مصاحبه چند مرحله دارد؟» – این سوال هم معلومات می‌دهد هم آرامش.

۴.۲ – مرحله ۱: Screening (مکالمه اول)

چه می‌گذرد: ۲۰-۳۰ دقیقه با recruiter یا لید – درباره‌ی تو (مسیر، تجربه، انتظار حقوق) و درباره‌ی آن‌ها (شغل، تیم).
چه می‌سنجند: ارتباط رزومه با شغل + ارتباط کلامی + انگیزه.

چیکار کنی: - «خودت را معرفی کن» را آماده داشته باش (۲ دقیقه – فصل ۱۰ قالب کامل دارد) - حقوق را در این مرحله می‌توانی بازه بگویی؛ یا بپرسی «رنج آگهی چیست؟» - ۲-۳ سوال بپرس (فصل ۱۱) – حتی در screening، سوال پرسیدن = امتیاز

جمله‌های کلیدی: «چرا این شرکت؟» → جوابت باید حاوی یک واقعیت درباره‌ی آن شرکت باشد (محصولش را دیده‌ای، مشکلتش را می‌فهمی) – نه «شرکت بزرگی است».

۴.۳ – مرحله ۲: Technical (سوالات فنی)

چه می‌گذرد: ۴۵-۶۰ دقیقه سوالات React/Next/JS/TS – معمولاً گفتگویی، گاهی با share screen و باز کردن کد.
چه می‌سنجند: عمق واقعی (نه حفظیات)، تجربه‌ی عملی، مرزهای دانشات.

چیکار کنی: - فصل‌های ۵ تا ۷ همین دوره = مخزن سوال‌ها – جواب‌ها را بلند تمرین کن - از تجربه‌ی واقعی مثال بزن: هر جواب فنی + یک «در پروژه‌ی X من...» = امتیاز طلایی - سوال ندانسته؟ قالب فصل ۱ (Thinking Out Loud + قدم ۵)

۴.۴ – مرحله ۳: Live Coding (کد زنده)

چه می‌گذرد: یک مساله کوچک را زنده می‌نویسی (در CodeSandbox/ادیتور اشتراکی) – مثلاً debounce، کامپوننت با fetch، فیلتر لیست.

چه می‌سنجند: فرایند فکر + عادت‌های کدنویسی + برخورد با گیر کردن – نه فقط جواب نهایی.

چیکار کنی: فصل ۸ همین دوره – ۸ تمرین رایج با کد کامل + پروتکل رفتار در حین کدنویسی (بلند فکر کن، سوال بپرس، تست با مثال).

۴.۵ – مرحله ۴: System Design (برای mid+)

چه می‌گذرد: «یک کامپوننت X را طراحی کن» یا «یک آپ Y چطور معماری می‌شود؟» – بدون کد، با دیاگرام و trade-off.

چه می‌سنجند: تفکر کلان، آگاهی از trade-off ها، سوال پرسیدن قبل از حل.

چیکار کنی: فصل ۹ – دو پاسخ نمونه کامل.

۴.۶ – مرحله ۵: Behavioral (رفتاری)

چه می‌گذرد: «تعارض را چطور حل کردی؟»، «یک شکست را بگو» – با لید/مدیر.

چه می‌سنجند: قابل‌همکاری بودن، مسئولیت‌پذیری، خودآگاهی.

چیکار کنی: فصل ۱۰ – ۴ + STAR جواب نمونه کامل آماده.

۴.۷ – جدول آمادگی سریع (قبل هر مصاحبه)

امشب این را تمرین کن	اگر فردا این مرحله است
معرفی ۲ دقیقه‌ای + تحقیق محصول شرکت + ۳ سوال	Screening
فصل ۵-۷ را بلند مرور کن؛ دوتا جواب React را کامل بگو	Technical
فصل ۸: دو تمرین با تایمر ۲۰ دقیقه	Live Coding
فصل ۹: یک طراحی کامل بلند بگو	System Design
۴ جواب STAR آماده + سوالات تو	Behavioral

جمع‌بندی فصل ✓

- شش مرحله: screening → technical → live coding → system design → behavioral → offer
- «فرایند چند مرحله دارد؟» = اولین سوال هر مصاحبه
- هر مرحله یک مهارت می‌سنجد – آماده‌سازی متفاوت می‌خواهد
- جدول ۴.۷ = برنامه شب قبل هر مرحله

تمرین فصل ۴ 📄

1. خودت را در موقعیت فرضی بگذار: سه مرحله‌ی بعدی‌ات را برای شرکت فرضی «دیجی‌استور» برنامه‌ریزی کن (چه تمرینی، کدام فصل).
2. جواب «چرا دیجی‌استور؟» را بنویس – با یک واقعیت واقعی از محصولش (برو سایش را ببین!).
3. سوال حقوق را در screening چطور باز می‌کنی؟ سه جمله بنویس (راهنما: فصل ۱۱).
4. برای هر مرحله، «بزرگ‌ترین خطای ممکن» را بنویس – خودآگاهی = نصف آمادگی.

▼ نمونه جواب تمرین ۴

سه مرحله‌ی من: (۱) Technical با سوالات React → امشب فصل ۶ را بلند تمرین می‌کنم؛ (۲) Live coding → فردا دو تمرین فصل ۸ با تایمر؛ (۳) Behavioral → جمعه ۴ جواب STAR را مرور می‌کنم.

جواب «چرا دیجی‌استور؟»: «سبد خرید و فیلترهای تان را تست کردم – تجربه موبایل‌تان عالی است ولی می‌دیدم صفحه جزئیات محصول CSR کامل است؛ با تجربه SSR/ISR در Next.js می‌توانم سئو و سرعت اولیه‌اش را بهتر کنم.»

خطاهای ممکن هر مرحله: screening = نپرسیدن هیچ سوال و گفتن حقوق خیلی دقیق؛ technical = جواب بلوف؛ live coding = سکوت و کدنویسی بی‌توضیح؛ behavioral = انتقاد از شرکت قبلی؛ نهایی = نگفتن حقوق و بستن بدون تاریخ.

➔ **فصل بعد:** سوالات JavaScript/TypeScript با جواب کامل – شروع بخش فنی!

فصل ۵ – ۱۸ سوال JavaScript/TypeScript با جواب کامل

🎯 هدف: پرتکرارترین سوالات JS/TS مصاحبه‌های فرانت – هر سوال: جواب کوتاه (۱۵ ثانیه‌ای) + نکته عمیق (وقتی بیشتر بخواهند). تمرین: جواب‌ها را بلند بگو، نه بخوان.

💡 الگوی جواب‌دهی: اول تعریف در یک جمله، بعد یک مثال یک‌خطی، بعد (اگر پرسیدند) trade-off.

۵.۱ – مبانی

Q1: فرق var، let و const؟

جواب کوتاه: function-scoped `var` و hoisted است (undefined قبل از تعریف) و می‌تواند دوباره تعریف شود. `let` و block-scoped `const` اند و TDZ دارند. `const` بازتخصیص ممنوع (ولی محتوای آبجکتش تغییر می‌کند).
نکته عمیق: در کد مدرن تقریباً همیشه `const` مگر بازتخصیص لازم – و `var` عملاً منسوخ است. سوال پشت سوال معمولاً: «hoisting چیست؟» → بالا آمدن تعریف به ابتدای scope در فاز کامپایل.

Q2: فرق = و ===

جواب کوتاه: `=` قبل مقایسه type coercion می‌کند (`1 === true` → `"1" = 1`) هم نوع هم مقدار را چک می‌کند.
نکته عمیق: دو استثنای معروف `null = undefined` → `true` و `NaN = NaN` → `false` (با خودش برابر نیست!). قاعده: همیشه `===` مگر knowingly با `null/undefined`.

Q3: Event Loop چیست؟ (پرسوالی‌ترین سوال JS!)

جواب کوتاه: JS single-threaded است؛ event loop صف callback ها (macrotask ها مثل `setTimeout`) و microtask ها (Promise ها) را می‌چرخد: کد sync تمام می‌شود → همه microtask ها → سپس یک macrotask → دوباره microtask ها...

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");
// خروجی: 1 2 3 4 ← microtask (3) قبل از macrotask (2)!
```

نکته عمیق: این است که `setTimeout(fn, 0)` فوری نیست – فقط «در صف بعدی macrotask». و چرا رندر UI گیر نمی‌کند وقتی درست بنویسی.

Q4: Closure چیست؟ مثال کاربردی؟

جواب کوتاه: تابعی که به متغیرهای scope پدرش — حتی بعد از پایان آن scope — دسترسی دارد.

```
function makeCounter() {  
  let count = 0; // private! دسترسی نیست  
  return () => ++count;  
}  
const inc = makeCounter();  
inc(); inc(); // 2 - count مانده است
```

کاربردهای واقعی: state خصوصی، debounce/throttle (تایمر داخل closure)، memoization. و تله معروفش در حلقه‌ها با `var` (همه همان count را می‌بینند؛ با `let` حل می‌شود).

Q5: Hoisting و TDZ؟

جواب کوتاه: تعریف‌ها (declarations) قبل اجرا به بالای scope منتقل می‌شوند. Function declaration ها کامل hoist می‌شوند؛ `var` فقط تعریفش (undefined)؛ hoist `let/const` می‌شوند اما تا خط تعریف در TDZ (Temporal Dead Zone) اند و دسترسی ارور ReferenceError می‌دهد.

۵.۲ — توابع و this

Q6: فرق arrow function و regular function؟

جواب کوتاه: سه فرق: `this` arrow خودش را ندارد (lexical — از enclosing می‌گیرد)، سازنده نیست (new نمی‌شود)، و `arguments` ندارد.

نکته عمیق: در event handler های کلاس قدیمی arrow لازم بود چون `this` را حفظ کند؛ در React مدرن با `function` component ها کمتر داغ است ولی در سوال ادامه دارد.

Q7: call, apply و bind؟

جواب کوتاه: هر سه `this` را کنترل می‌کنند: `call(fn, thisArg, a, b)` آرگومان‌ها پشت هم؛ `apply` آرایه‌ای؛ `bind` نسخه‌ی جدید بایندشده برمی‌گرداند (اجرا نمی‌کند).

مثال: `const log = console.log.bind(console)` — الگوی کلاسیک.

Q8: Prototype و inheritance؟

جواب کوتاه: هر آبجکت JS یک `[[Prototype]]` دارد؛ جستجوی property اگر پیدا نشود به پروتوتایپ پدر می‌رود و بالاتر (prototype chain) تا `null`. کلاس‌های ES6 شیرین‌سازی همین مکانیزم‌اند.

مثال کلاسیک: `map` – `const arr = []; arr.map(...)` در `Array.prototype` است، نه خود `arr`.

۵.۳ – Async

Q9: Promise و state هایش؟

جواب کوتاه: آبجکتی نماینده‌ی نتیجه‌ی آینده‌ی عملیات `async` – سه `state` دارد: `pending` → `fulfilled` / `rejected` (و یک‌بار برای همیشه قفل می‌شود). با `.then/.catch/.finally` یا `async/await` مصرف می‌شود.

Q10: `async/await` چگونه کار می‌کند؟ `error handling`؟

جواب کوتاه: `async` تابعی که همیشه `Promise` برمی‌گرداند؛ `await` توقف منطقی تا `resolve` (بدون بلاک کردن `thread` – زیر `hood` همان `Promise chain` است). خطا با `try/catch` گرفته می‌شود.

```
async function load() {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch 404 روی throw نمی‌کند
    return await res.json();
  } catch (e) { /* شبکه http یا */ }
}
```

نکته عمیق: `await Promise.all([a, b])` برای اجرای موازی – سریالی `await` کردن دو `fetch` = `waterfall` (فصل ۷ مرور `Next` هم بود!).

Q11: `Promise.all` در برابر `allSettled` و `race`؟

جواب کوتاه: `all` = همه موفق یا اولین `reject` (`fail-fast`)؛ `allSettled` = همه را منتظر می‌ماند و نتیجه هرکدام را جدا می‌دهد؛ `race` = اولین `settle` (موفق یا شکست)؛ `any` = اولین موفق.

کاربرد واقعی: لیست چند `API` مستقل → `allSettled` (یکی `fail` شد بقیه را نشان بده)؛ `timeout` دستی → `race` با تایمر.

۵.۴ – ساختار داده و الگوها

Q12: `deep copy` و `shallow`؟

جواب کوتاه: `shallow` فقط سطح اول را کپی می‌کند (`{...obj}`) – آبجکت‌های تودرتو هنوز مرجع مشترک دارند؛ `deep copy` کل درخت را (`structuredClone(obj)`) استاندارد جدید).

تله مصاحبه: `{...user.address}` - بدون دقت، تغییر `copy.address.city` اصل را هم

عوض می‌کند!

Q13: debounce و throttle؟ (پیاده‌سازی بخواهند!)

جواب debounce:

```
function debounce(fn, delay = 300) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

جواب throttle:

```
function throttle(fn, interval = 300) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= interval) {
      last = now;
      fn.apply(this, args);
    }
  };
}
```

تفاوت: debounce = «بعد از سکوت اجرا کن» (سرچ); throttle = «حداکثر هر X ms یک بار» (اسکرول، resize). closure ها را که بلدی (Q4) هر دو را زنده می‌نویسی.

Q14: map, filter, reduce – و reduce پیچیده؟

جواب کوتاه: map تبدیل، filter فیلتر، reduce تاخوردن به یک مقدار. reduce برای groupBy و tally:

```
const byCat = items.reduce((acc, item) => {
  (acc[item.category] ??= []).push(item);
  return acc;
}, {});
```

نکته: `(acc[k] ??= [])` = «اگر نیست بساز» - با operator جدید `??` خوش‌دست.

Q15: فرق null و undefined و optional chaining و nullish؟

جواب کوتاه: undefined = «مقدار داده نشده» (سیستم); null = «عمداً خالی» (برنامه‌نویس). `a?.b` = اگر `a` null/undefined نبود برو جلو؛ `a ?? b` = فقط برای null/undefined جایگزین کن (برخلاف `||` که 0 و "" را هم می‌گیرد!).

تله کلاسیک: `count || 10` وقتی `count = 0` است → `!!count` درست: `count ?? 10`.

TypeScript – ۵.۵

type و interface چه فرقی دارند؟

جواب کوتاه: اکثراً همپوشان؛ interface قابلیت declaration merging دارد (تعریف دوباره = ادغام) و برای شکل آبجکت‌ها مرسوم است؛ type union/intersect/primitive و utility ها را هم می‌گیرد. قاعده تیم: یکی را انتخاب و یکدست کن.

Q17: generic چیست و کجا به کار می‌آید؟

جواب کوتاه: تابع/نوعی که با type پارامتری می‌شود و تایپ‌سیف می‌ماند:

```
function first<T>(arr: T[]): T | undefined {  
  return arr[0];  
}  
const n = first([1, 2]); // n: number | undefined – خودکار!
```

مثال واقعی: `useState<TState, TFormData>` در React 19 – generics همه جا هستند.

Q18: unknown و never؟

جواب کوتاه: `any` = چک تایپ خاموش (خطرناک)؛ `unknown` = همان انعطاف ولی مصرف‌کننده مجبور به narrow شدن است (امن)؛ `never` = هیچ مقداری (توابعی که همیشه throw یا حلقه بی‌نهایت) – برای exhaustiveness check هم کاربرد دارد.

جمله طلایی: «any را در کد production امضا نمی‌کنم – unknown بگیر و narrow کن.»

جمع‌بندی فصل ✓

- Event loop، closure، debonuce/throttle = سه سوال ستاره‌ی مصاحبه JS
- snapshot شدن state (فصل React) ریشه‌اش همین event loop و صف است
- `??` به جای `||` برای nullish واقعی؛ `unknown` به جای `any`
- هر جواب = یک جمله تعریف + مثال یک خطی + آمادگی برای سوال عمیق‌تر

تمرین فصل ۵ (بلند بگو – ضبط کن!) 📝

1. سه سوال Q1، Q3، Q9 را با ضبط صدا جواب بده و بشنو – کجاها گنگ است را اصلاح کن.
2. بدون نگاه به کتاب، debounce را بنویس (تایمر ۵ دقیقه) – بعد مقایسه کن.

3. خروجی این کد را حدس بزن، بعد اجرا کن:

```
console.log("a");
setTimeout(() => console.log("b"));
Promise.resolve().then(() => console.log("c"));
console.log("d");
```

4. تابع `groupBy(items, keyFn)` با `reduce` بنویس و برای `orders` به تفکیک `status` تست کن.

5. تفاوت `obj.count || 5` و `obj.count ?? 5` را با سه مقدار `count = 0`، `5`، `undefined` نشان بده.

▼ جوابها

3. خروجی: `sync – a d c b` اول `(a, d)`، بعد `(c)` `microtask`، بعد `(b)` `macrotask`.

4.

```
function groupBy<T>(items: T[], keyFn: (item: T) => string): Record<string, T[]> {
  return items.reduce((acc: Record<string, T[]>, item) => {
    const key = keyFn(item);
    (acc[key] ??= []).push(item);
    return acc;
  }, {});
}
```

5. `||` → `count=0` می‌دهد ۵ (غلط!)، `??` می‌دهد ۰؛ `count=5` → هر دو ۵؛ `count=undefined` → هر دو ۵.

→ فصل بعد: ۲۰ سوال React – میدان اصلی تو.

فصل ۶ – ۲۰ سوال React با جواب کامل

🎯 هدف: میدان اصلی مصاحبه‌ی فرانت‌اند تو. هر سوال: جواب مصاحبه‌ای + نکته عمیق. از فصل ۱-۴ مرور Next هم پشتیبانی می‌کنیم (چون سوال‌ها گاهی React خالص، گاهی در Next).

۶.۱ – مبانی

Q1: Virtual DOM چیست و چرا؟

جواب: React یک نمایش سبک DOM را در حافظه نگه می‌دارد؛ هنگام تغییر state، درخت جدید با قبلی مقایسه می‌شود (reconciliation/diffing) و فقط قسمت‌های تغییرکرده به DOM واقعی اعمال می‌شوند – چون دستکاری DOM گران‌ترین عملیات مرورگر است.

عمیق‌تر: الگوریتم $O(n)$ با فرض‌های React: عناصر هم‌نوع مقایسه می‌شوند؛ key ها هویت آیتم‌های لیست را می‌دهند.

Q2: رندر و re-render کی رخ می‌دهد؟

جواب: رندر = صدا زدن تابع کامپوننت. re-render وقتی: state عوض شود، والد re-render شود (بدون memo)، یا context مصرفی تغییر کند.

نکته: re-render یعنی اجرای دوباره تابع – نه لزوماً تغییر DOM (diff بعدش تصمیم می‌گیرد). StrictMode دو بار صدا می‌زند برای کشف ناخالصی.

Q3: چرا key و چرا index بد است؟

جواب: key هویت آیتم را بین رندها به React می‌دهد تا diff درست انجام شود. index وقتی لیست reorder/insert می‌شود، آیتم اشتباه را به key قبلی می‌چسباند → state داخلی آیتم‌ها (input ها!) جابجا می‌شود و diff ناکارآمد می‌شود.

Q4: props drilling چیست و راه‌حل‌ها؟

جواب: پاس دادن props از لایه‌های واسط بی‌استفاده. راه‌حل‌ها به ترتیب: composition (children) – معمولاً کافی است! → Context (برای داده کم‌تغییر سراسری) → ابزار state سراسری (Zustand/TanStack) برای state سروری.

Q5: useState و snapshot semantics؟

جواب: state در هر رندر یک snapshot ثابت است؛ آپدیت وابسته به مقدار قبلی $\rightarrow$ فرم تابعی `setX(x  $\Rightarrow$  x+1)`؛ چند `setState` در یک event batch می‌شود. (فصل ۲ مرور Next کاملش را دارد).

Q6: useEffect دقیقاً برای چیست و چه چیزهایی نیست؟

جواب: همگام‌سازی با سیستم‌های بیرونی (DOM، subscription، غیرری‌اکتی، تایمر) — با `cleanup`. نیست: محاسبه مشتق‌شده (در رندر)، event handling (در handler)، `fetch` داده در اپ‌های جدی (فریم‌ورک/Query).

نکته: وابستگی‌ها صادقانه؛ `cleanup` قبل هر اجرای دوباره؛ `race condition` با `ignore flag` یا `AbortController`.

Q7: useMemo/useCallback کی؟

جواب: `useMemo` محاسبه سنگین یا مرجع پایدار برای وابستگی؛ `useCallback` خود تابع برای `memoized children` یا `dependency`. بدون اندازه‌گیری نزن — React Compiler در راه است.

Q8: فرق controlled و uncontrolled component؟

جواب: `controlled: value` از `state` می‌آید (`onChange  $\Rightarrow$  setState` — single source of truth)؛ `uncontrolled: DOM` خودش نگه می‌دارد و با `ref` می‌خوانی (مثلاً `<input defaultValue>` یا `file input`).

انتخاب: فرم ساده/فایل $\rightarrow$ `uncontrolled` سبک‌تر؛ اعتبارسنجی زنده/شرط‌ها $\rightarrow$ `controlled`. (در `Next.js` فرم‌ها با `Server Action` هم می‌روند).

Q9: Context چه وقت anti-pattern است؟

جواب: برای `state` پرتغییر پرتکرار (مثل مقدار `input`) — هر تغییری همه‌ی مصرف‌کننده‌ها را `re-render` می‌کند. راه: `state` را نزدیک مصرف‌کننده نگه دار، `Context` را به چند `Context` کوچک بشکن، یا `state` سروری را با `Query` مدیریت کن.

۶.۳ — معماری و پرفورمنس

Q10: React.memo و کامپوننت‌ها؟

جواب: `memo(Component)` رندر مجدد را وقتی `props` بدون تغییرند حذف می‌کند — فقط وقتی: رندر سنگین است + والدش زیاد `re-render` می‌شود + `props` پایدار پاس می‌دهند (پس `useCallback/useMemo` همراهش لازم است). سه شرط باهم، وگرنه بی‌فایده.

Q11: key-based reset

جواب: ترفند مستند: برای reset کامل یک کامپوننت (فرم!)، به آن `key={resetCounter}` بده — تغییر `key =` `unmount/remount` و همه state اول می‌شود. تمیزتر از reset کردن ده state دستی.

Q12: lifting state در برابر state سراسری؟

جواب: اول lifting (نزدیک‌ترین جد مشترک)؛ اگر درخت عمیق شد composition؛ اگر سروری است و چند صفحه دارد → TanStack Query (cache + retry + stale logic آماده)؛ client-side سراسری واقعی (تم/سشن) → Context/Zustand.

Q13: error boundary چیست؟

جواب: کامپوننتی که ارورهای رندر فرزندان را می‌گیرد (در React با `componentDidCatch/useEffect+throw` در Next از `error.tsx` استفاده می‌کند) — از فاجعه‌ی «سفیدشدن کل اپ» جلوگیری می‌کند و بخش خطادیده را ایزوله می‌کند. ارورهای event handler را نمی‌گیرد (try/catch دستی).

Q14: Suspense چه می‌کند؟

جواب: به کامپوننت‌های «معلق» (promise که با `use` خوانده می‌شود، یا lazy) یک `fallback` می‌دهد — رندر بدون بلاک ادامه می‌یابد و وقتی آماده شد جایگزین می‌شود. در Next.js با `streaming` همان است (فصل ۹ مرور).

۶.۴ — React 19 و مدرن

Q15: React 19 چه چیزهای جدیدی آورد؟

جواب: هوک‌های فرم (`useActionState/useFormStatus/useOptimistic`)، هوک `use` (خواندن `promise/context` با شرط)، `ref` به عنوان `prop` معمولی (`goodbye forwardRef`)، بهبود پیام‌های `hydration`، و پایهی `Server Components/Actions` (که در Next مصرف می‌کنیم).

Q16: Server Component چیست و فرقی با SSR کلاسیک؟

جواب: SSR کلاسیک فقط رندر اولیه HTML بود و بعد کل JS می‌آمد hydrate شود. Server Component کدی است که فقط روی سرور اجرا می‌شود و `payload` رندر شده می‌فرستد — JS آن هرگز به کلاینت نمی‌رود؛ می‌تواند مستقیم دیتابیس بخواند. Client Component ها (`use client`) برای `interactivity` هستند و در سرور هم `pre-render` می‌شوند.

Q17: چرا key در لیست ولی در فرم هم می‌گذارند؟

جواب (سوال چرخشی): همان منطق هویت – برای «force remount» یک کامپوننت (مثلاً خالی کردن فرم بعد از submit) `key={version}` عوض می‌کنی تا React آن را از نو بسازد؛ تمیزتر از دستی صفر کردن `state`.

Q18: چطور یک لیست بزرگ ۱۰هزار آیتمی را رندر می‌کنی؟

جواب: virtualization – فقط آیتم‌های داخل viewport رندر می‌شوند (react-window / TanStack Virtual)؛ همراه memo روی ردیف‌ها. اگر داده از سرور است: pagination/cursor + infinite scroll. هرگز ۱۰هزار DOM node نریز.

Q19: StrictMode چه می‌کند و چرا دو بار رندر؟

جواب: در dev، دوبار صدا زدن کامپوننت‌ها/effects برای لو رفتن side-effect ها و ناخالصی. production حذف است. دیدن دوباره‌ی لاگ‌ها باگ نیست – سیگنال اصلاح است.

Q20: state را کجا نگه داری: useState، URL، سرور؟

جواب (سوال سطح بالا): فیلترها و صفحه‌ها → URL (shareable – searchParams و back-safe)؛ UI موقت (modal باز) → useState؛ داده‌ی سرور → TanStack Query یا fetch در Server Component (فصل ۷ مرور Next). هر داده یک خانه‌ی درست دارد – duplicating state را حذف کن.

جمع‌بندی فصل

- هسته: effects، snapshot، key، VDOM/reconciliation همگام‌ساز
- پرفورمنس: memo سه‌شرطی، Context، virtualization ضدالگو
- React 19: فرم‌هوک‌ها، Server Components، use.
- هر جواب + «در پروژه‌ی من...» = امتیاز

تمرین فصل ۶

1. ده سوال از این فصل را انتخاب و با ضبط صدا جواب بده (هر کدام زیر ۶۰ ثانیه).
2. Q18 را عملی کن: یک لیست ۵۰۰۰ آیتمی بساز، بدون virtualization benchmark بگیر، بعد با virtualization اعداد را یادداشت کن (در مصاحبه عدد طلایی است!).
3. سوال چرخشی Q13 را آماده کن: «error boundary در Next.js چطور؟» → error.tsx (فصل ۹ مرور).
4. مصاحبه‌گر می‌گوید «useEffect را دوست ندارم» – منظورش چیست و جوابت؟ (راهنما: You Might Not Need an Effect)

▼ جواب نمونه تمرین ۴

یعنی با استفاده‌های اشتباه effect جنگیده (محاسبه در event، effect در chain، effect های fetch). جوابت: «موافقم – اکثر useEffect هایی که می‌بینم جای دیگری belong می‌کنند: محاسبه در رندر، رویداد در handler، و fetch سروری در Server Component. Effect را من فقط برای همگام‌سازی بیرونی نگه می‌دارم – subscription ها و ...» + یک مثال refactor واقعی از پروژه‌ات.

→ **فصل بعد:** ۱۶ سوال Next.js – خانه دوم تو.

فصل ۷ – ۱۶ سوال Next.js با جواب کامل

🎯 هدف: میدان تخصص تو – سوالاتی که برای موقعیت Next.js واقعاً پرسیده می‌شود (Next 15/16، App Router). هر سوال: جواب مصاحبه‌ای + عمیق‌تر. مرور فصل‌های ۵-۱۰ دوره Next پشتیبان این فصل است.

۷.۱ – App Router و رندر

Q1: فرق App Router و Pages Router؟

جواب: App Router (پوشه app) با React Server Components بومی شده: پیش‌فرض سرور، layout های تو در تو با state پایدار، streaming/Suspense یکپارچه، Server Actions، و data fetching ساده با Pages. Router (پوشه pages) مدل قدیمی getServerSideProps/getStaticProps است. پروژه‌های جدید: App Router.

Q2: SSG، SSR، ISR – و کی هرکدام؟

جواب: SSG در build یک‌بار (لندینگ/مستندات)؛ ISR در build + بازتولید دوره‌ای (`revalidate = N`) – بلاگ/فروشگاه؛ SSR هر درخواست (داده شخصی/سشن). در App Router: پیش‌فرض static تا وقتی چیزی dynamic نشود (cookies/headers/searchParams/no-store).

Q3: چگونه یک صفحه را dynamic یا static نگه می‌داریم؟

جواب: اعلام قصد: `revalidate` برای تازگی دوره‌ای؛ استفاده از `cookies()/headers()/searchParams` یا `fetch no-store` صفحه را dynamic می‌کند؛ `generateStaticParams` مسیرهای دینامیک را در build می‌سازد؛ و `dynamic = "force-dynamic"` برای اجبار.

Q4: Server Component چیست و کی Client لازم است؟

جواب: پیش‌فرض سرور: کدش به کلاینت نمی‌رود، دیتابیس/secret مستقیم، async مستقیم. Client Component (`"use client"`) برای state/effect/رویداد/مرورگر API – و اولین رندرش هم SSR است. مرز یک‌طرفه به پایین است و داده فقط serializable رد می‌شود (تابع ممنوع به‌جز Server Actions).

Q5: چرا params یک Promise است؟

جواب (Next 15+): برای سازگاری با استریم و async rendering – پارامترها ممکن است در آینده async حل شوند؛ پس `const { slug } = await params` (در کلاینت با `use(params)`).

Q6: streaming چیست و چگونه؟

جواب: HTML پویا فوری ارسال و بخش‌های کند (کوئری سنگین) با `<Suspense>` بعداً تزریق می‌شوند – به جای بلاک شدن کل صفحه. `loading.tsx` همان `Suspense` خودکار مسیر است. `FCP` و `TTFB` عالی می‌شود.

Q7: چرا این صفحه‌ها static نمی‌شود؟

جواب (دییباگ رایج!): یکی از این‌ها: `searchParams` خوانده شده، `fetch`، `cookies()/headers()` با `no-store`، یا `random/Math.now` در رندر. با `next build` بین فلگ `○ (static)` یا `(dynamic)` چیست – `build` گزارش هر مسیر را می‌دهد.

۷.۲ – داده و mutation

Q8: Data fetching در App Router چگونه انجام می‌شود؟

جواب: مستقیم `await fetch(...)` در `Server Component` (بدون `useEffect`) با آپشن‌های کش `{ next: { revalidate, tags } }`؛ `Request Memoization` یک `URL` را در یک رندر یک‌بار می‌زند؛ `fetch` های مستقل را `Promise.all` موازی کن (ضد waterfall)؛ داده کند را پایین‌تر ببر و استریم کن.

Q9: Server Action چیست و چه فرقی با API Route دارد؟

جواب: تابع `async` با `"use server"` که کلاینت مستقیم صدا می‌زند (RPC) – برای فرم‌ها و `mutation` های UI خود سایت، با یکپارچگی فرم و `route.ts` (`Route Handler`، `progressive enhancement`) برای API عمومی: مصرف‌کننده خارجی، `webhook`، کنترل کامل هدر/`status`.

Q10: بعد از داده mutation چگونه تازه می‌شود؟

جواب: `revalidatePath("/blog")` یا `revalidateTag("posts")` داخل همان `Server Action` – کش مربوطه باطل و رندر بعدی تازه. جفتش با `fetch(..., { next: { tags: [...] } })`.

Q11: how do you handle form validation (فرم + اعتبارسنجی)

جواب: کلاینت: UX (فوری)؛ سرور: خط دفاع واقعی – `zod` در `Server Action` با `useActionState` برای برگرداندن ارور؛ هرگز به `validation` کلاینت اعتماد نکن؛ و `authorization` (صاحب رکورد) هم در همان اکشن.

Q12: کجا داده state نگه می‌داری؟ (URL در برابر useState)

جواب: فیلتر/صفحه/تب → URL (searchParams — shareable، back-safe، SEO)؛ UI گذرا (مودال) → useState؛ داده سرور → fetch/Query. الگوی فیلتر: `router.push(?q=...)` و سرور رندر می‌کند.

۷.۳ — بهینه‌سازی و متادیتا

Q13: next/image چه کار می‌کند و تنظیمات مهمش؟

جواب: resize/فرمت مدرن (WebP/AVIF)، lazy، on-demand، پیش‌فرض، CLS صفر (اجبار ابعاد)، `priority` برای `sizes`، LCP، برای انتخاب منبع درست، `remotePatterns` برای دامنه‌های خارجی، `placeholder="blur"` با `static` `import`.

Q14: Metadata و سئو در App Router؟

جواب: `export const metadata` استاتیک و `generateMetadata` دینامیک (از دیتابیس) — `title template`، `canonical`، `openGraph`. فایل‌های قراردادی: `sitemap.ts`، `robots.ts`، `opengraph-image.tsx` (OG با کد تولید می‌شود!). Yoast/دیتابیس → همان کوئری می‌شود `metadata`.

Q15: چطور Next.js اپ را امن نگه می‌دارد و تو چه می‌کنی؟

جواب: Next: فرار خودکار خروجی، پشتیبانی Server Actions، CSP بدون endpoint عمومی، `NEXT_PUBLIC_` تنها راه لو دادن به باندل. من: رازها فقط سرور، validation سروری با `zod`، `authorization`، `headers`، `mutation` امنیتی در `middleware/next.config`.

Q16: یک اپ Next.js را چطور بهینه برای Core Web Vitals می‌کنی؟

جواب: LCP: تصویر hero با `priority` + فونت با `next/font` (بدون درخواست خارجی) + استریم پوسته؛ CLS: ابعاد تصویر + `skeleton`؛ INP: جزایر کلاینت کوچک + `transitions` برای آپدیت‌های سنگین؛ داده: `Promise.all` + استریم؛ کش: `ISR/revalidateTag` درست.

جمع‌بندی فصل ✓

- رندر: `static` پیش‌فرض؛ قصد را اعلام کن (`revalidate/no-store/cookies`)
- داده: `fetch` سروری + `streaming` + `Promise.all` + `memoization`؛ `mutation` با `revalidateTag` + `Server Action`
- RSC: مرز `serializable`؛ `Promise`؛ `params`؛ `Client` جزیره‌ای

تمرین فصل ۷

1. ۱۶ سوال را با ضبط صدا جواب بده – سوال‌های Q2، Q4، Q9، Q10 بیشترین فراوانی را دارند.
2. سناریو: مصاحبه‌گر می‌گوید «چرا Server Action به جای API معمولی برای فرم؟» – جوابت را با یک trade-off واقعی بده (بدون فصل ۸ مرور Next نگاه کن!).
3. «لیست محصولات ۱۰هزارتایی با فیلتر و جستجو» – معماری رندر و داده را طرح کن (URL؟ SSG/ISR/SSR؟ pagination؟).
4. از ریپوی یکی از پروژه‌هایت، `next build` بزن و گزارش static/dynamic هر مسیر را تفسیر کن.

▼ جواب نمونه تمرین ۲ و ۳

تمرین ۲: «برای mutation های UI خود سایت، Server Action یک لایه RPC تایپ‌سیف بدون endpoint عمومی می‌دهد: فرم مستقیم به تابع وصل است، revalidateTag کنار ذخیره است و progressive enhancement هم دارد. اما برای مصرف‌کننده خارجی (اپ موبایل/webhook) Route Handler می‌سازم – اعتبارسنجی zod را مشترک نگه می‌دارم تا DRY باشد.»

تمرین ۳: لیست = ISR با ۶۰ revalidate و tags: ["products"]؛ فیلتر/جستجو با searchParams (dynamic) و cursor pagination؛ جزئیات = generateStaticParams برای محبوب‌ها + dynamicParams برای بقیه با ISR؛ mutation ادمین = Server Action + revalidateTag("products")؛ تصاویر next/image با remotePatterns.

→ **فصل بعد:** ۸ تمرین لایو-کدینگ با کد کامل.

فصل ۸ – ۸ تمرین Live Coding با کد کامل جواب

🎯 هدف: رایج‌ترین تمرین‌های زنده مصاحبه فرانت – هر تمرین: صورت مساله + نکات صحبت‌کردن + کد کامل. با تایمر ۲۰ دقیقه تمرین کن و بلند حرف بزن.

🗨️ پروتکل طلایی لایو-کدینگ: ۱. قبل از کد: سوال شفاف‌سازی بپرس («ورودی همیشه آرایه است؟ خالی ممکن است؟») ۲. مسیر را بلند بگو («اول state، بعد fetch با loading، بعد...») ۳. ساده شروع کن، بعد بهبود بده («نسخه ساده، بعداً virtualization») ۴. حین کد: هر بلوک را یک جمله توضیح بده ۵. آخر: خودت با مثال تست کن و edge case بگو

تمرین ۱ – Debounce Hook (پرتکرارترین!)

صورت مساله: «یک useDebounce بساز که ورودی جستجو با تاخیر ۳۰۰ms اعمال شود.»

```
import { useEffect, useState } from "react";

function useDebounceValue<T>(value: T, delay = 300): T {
  const [debounced, setDebounced] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id); // هر تایپ جدید، تایمر قبلی را لغو می‌کند
  }, [value, delay]);

  return debounced;
}

// مصرف:
function Search() {
  const [query, setQuery] = useState("");
  const debounced = useDebounceValue(query);

  useEffect(() => {
    if (debounced) fetchResults(debounced);
  }, [debounced]);
}
```

بگو: cleanup برای لغو تایمر قبلی – بدون آن هر تایپ یک fetch می‌سازد. و fetch هم race دارد (فصل ۳ مرور: ignore flag یا AbortController).

تمرین ۲ – لیست با fetch و سه حالت

صورت مساله: «لیست کاربران از API با حالت‌های loading/error/empty/error.»

```
function UserList() {
  const [users, setUsers] = useState<User[]>([]);
  const [state, setState] = useState<"loading" | "error" | "ready">("loading");

  useEffect(() => {
    let ignore = false;
    fetch("/api/users")
      .then((r) => {
        if (!r.ok) throw new Error(`HTTP ${r.status}`);
        return r.json();
      })
      .then((data) => { if (!ignore) { setUsers(data); setState("ready"); } })
      .catch(() => { if (!ignore) setState("error"); });
    return () => { ignore = true };
  }, []);

  if (state === "loading") return <Skeleton />;
  if (state === "error") return <ErrorMessage onRetry={() => location.reload()} />;
  if (users.length === 0) return <EmptyState />; // empty state نكن را فراموش نكن!

  return <ul>{users.map((u) => <li key={u.id}>{u.name}</li>)}</ul>;
}
```

بگو: چهار حالت (loading/error/empty/ready) – اکثر کاندیدها سه‌تای اول را یادشان می‌رود. `ignore` برای `race` `.condition`

تمرین ۳ – TODO List کامل

صورت مساله: «CRUD ساده: افزودن، تیک‌زدن، حذف، فیلتر».

```
type Todo = { id: number; text: string; done: boolean };

function Todos() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [filter, setFilter] = useState<"all" | "active" | "done">("all");

  const add = (text: string) =>
    setTodos((t) => [...t, { id: Date.now(), text, done: false }]); // فرم تابعی!
  const toggle = (id: number) =>
    setTodos((t) => t.map((x) => (x.id === id ? { ...x, done: !x.done } : x)));
  const remove = (id: number) => setTodos((t) => t.filter((x) => x.id !== id));

  const visible = todos.filter((t) =>
    filter === "all" ? true : filter === "done" ? t.done : !t.done
  );

  return (
    <
      <AddForm onAdd={add} />
      <ul>{visible.map((t) => (
        <li key={t.id}>
          <input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} />
          {t.text} <button onClick={() => remove(t.id)}>x</button>
        </li>
      ))}</ul>
      <FilterBar value={filter} onChange={setFilter} />
    </>
  );
}
```

بگو: immutable update (spread/map/filter)، فرم تابعی `setState`، و اینکه `visible` را محاسبه کردم نه state جدا (single source of truth).

تمرین ۴ – Accordion

صورت مساله: «لیست سوالات FAQ – یکی باز، بقیه بسته؛ با کلیک دوباره بسته شود.»

```
function Accordion({ items }: { items: { q: string; a: string }[] }) {
  const [openIndex, setOpenIndex] = useState<number | null>(null); // فقط index!

  return items.map((item, i) => (
    <div key={i}>
      <button
        onClick={() => setOpenIndex(openIndex === i ? null : i)}
        aria-expanded={openIndex === i} // امتیاز می‌آورد a11y!
      >
        {item.q}
      </button>
      {openIndex === i && <p>{item.a}</p>}
    </div>
  ));
}
```

بگو: به جای آرایه open ها، یک index – چون «یکی باز» یعنی state تک‌مقداری است. `aria-expanded` برای دسترس‌پذیری. و نسخه چندبازشو: `Set<number>`.

تمرین ۵ – Timer (شمارش معکوس)

صورت مساله: «تایمر ۱۰ ثانیه‌ای که به صفر که رسید متوقف شود.»

```
function Countdown({ from = 10 }: { from?: number }) {
  const [left, setLeft] = useState<number>(from);

  useEffect(() => {
    if (left === 0) return;
    const id = setInterval(() => setLeft(s => Math.max(0, s - 1)), 1000);
    return () => clearInterval(id); // پاک شود unmount در هر تغییر و
  }, [left !== 0]); // نمی‌سازد interval، ترفند: با رسیدن به صفر

  return <span>{left}</span>;
}
```

بگو: cleanup حیاتی است (گر نشود interval های همزمان انباشته می‌شوند). فرم تابعی چون وابسته به مقدار قبلی است. نسخه حرفه‌ای‌تر: `setInterval` یک‌بار + چک داخلش.

تمرین ۶ – Infinite Scroll

صورت مساله: «لیست پست‌ها که با اسکرول به انتها صفحه بعدی لود شود.»

```
function Feed() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [cursor, setCursor] = useState<string | null>(null);
  const [hasMore, setHasMore] = useState(true);

  const sentinelRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    const node = sentinelRef.current;
    if (!node || !hasMore) return;
    const obs = new IntersectionObserver([entry] => {
      if (entry.isIntersecting) loadMore();
    });
    obs.observe(node);
    return () => obs.disconnect();
  }, [hasMore, cursor]);

  async function loadMore() {
    const res = await fetch(`/api/posts?cursor=${cursor ?? ""}`);
    const { items, nextCursor } = await res.json();
    setPosts(p => [...p, ...items]); // فرم تابعی - با داده جدید ادغام
    setCursor(nextCursor ?? null);
    setHasMore(!!nextCursor);
  }

  return (
    <div>
      {posts.map((p) => <Card key={p.id} post={p} />)}
      {hasMore && <div ref={sentinelRef} /> }
    </div>
  );
}
```

بگو: cursor pagination (نه page number – چون insert جدید باعث تکرار می‌شود) + IntersectionObserver (بهتر از scroll listener که هر اسکرول فایر می‌شود).

تمرین ۷ – Form با اعتبارسنجی زنده

صورت مساله: «فرم ثبت‌نام: ایمیل معتبر، پسورد ۸+ کاراکتر، نمایش خطا زیر فیلد، دکمه disable تا معتبر.»

```
function Signup() {
  const [form, setForm] = useState({ email: "", password: "" });
  const [touched, setTouched] = useState({ email: false, password: false });

  const errors = {
    email: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email) ? null : "ایمیل نامعتبر",
    password: form.password.length ≥ 8 ? null : "حداقل ۸ کاراکتر",
  };
  const isValid = !errors.email && !errors.password;

  return (
    <Form onSubmit={e => { e.preventDefault(); isValid && submit(form); }}>
      <input value={form.email}
        onChange={e => setForm({ ...form, email: e.target.value })}
        onBlur={() => setTouched((t) => ({ ...t, email: true }))} />
      {touched.email && errors.email && <p className="err">{errors.email}</p>}
      {/* پسورد مشابه */}
      <button disabled={!isValid}>ثبت نام</button>
    </Form>
  );
}
```

بگو: validation = محاسبه مشتق شده در رندر (نه state جدا – single source of truth)؛ `touched` تا خطا از اول تایپ نمایش داده نشود؛ واقعی ترش: zod + سرور.

تمرین ۸ – Grid فیلترشده (ترکیبی – چاشنی نهایی)

صورت مساله: «لیست محصولات با سرچ و فیلتر دسته، همگام با URL».

```
import { useSearchParams, useRouter, usePathname } from "next/navigation";

function Products({ products }: { products: Product[] }) { // از Server Component داده از
  const router = useRouter();
  const pathname = usePathname();
  const params = useSearchParams();
  const q = params.get("q") ?? "";
  const cat = params.get("cat") ?? "all";

  function update(key: string, value: string) {
    const next = new URLSearchParams(params);
    value ? next.set(key, value) : next.delete(key);
    router.push(`${pathname}?${next}`);
  }

  const visible = products.filter((p) =>
    p.title.includes(q) && (cat === "all" || p.category === cat)
  );

  return (
    <input defaultValue={q}
      onChange={e => update("q", e.target.value)} />
    {/* ... فیلتر دسته ... */}
    {visible.map((p) => <Card key={p.id} product={p} />)}
  );
}
```

بگو: فیلتر در `URL → shareable`، `back-safe`، رندر سروری دوباره؛ `(uncontrolled)` `defaultVaue` تا `URL` منبع حقیقت باشد و `state` تکراری نسازم؛ در پروژه واقعی `debounce` هم می‌گذارم (تمرین ۱!) — اتصال دو تمرین = امتیاز بزرگ.

جمع‌بندی فصل ✓

- هشت تمرین پوشش می‌دهند: `hook` سفارشی، `fetch`/حالت‌ها، `state`، `CRUD`، `interval/cleanup`، `IntersectionObserver`، فرم مشتق‌شده، `URL-as-state`
- همیشه: سوال شفاف‌سازی → بلند فکر → ساده شروع → `edge case` → تست با مثال
- امتیازهای مکمل: `immutable update`، `empty state`، `a11y`، فرم تابعی `setState`

تمرین فصل ۸ 📄

1. هشت تمرین را با تایمر ۲۰ دقیقه‌ای حل کن (هر روز دو تا، هفته‌ای ۴ روز) — **بلند حرف بزن**.
2. تمرین ۲ را با `AbortController` بهتر کن (لغو `fetch` واقعی، نه فقط `ignore`).
3. تمرین ۶ را با لغو درخواست قبلی کامل کن (کاربر اسکرولش سریع است!).
4. چالش ترکیبی: (۶) `infinite scroll` + (۱) `debounce` در یک کامپوننت جستجوی زنده.
5. یک تمرین از مصاحبه واقعی (آنلاین سرچ کن "frontend live coding interview task") پیدا و حل کن و به این لیست اضافه کن.

▼ چک‌لیست روز مصاحبه لایو-کدینگ

- [] ادیتور را قبل شروع کن `figur` کن (فونت بزرگ‌تر، افزونه‌ها)
- [] سوال را بازگو کن + یک سوال شفاف‌سازی
- [] ساختار را بلند بگو قبل از تایپ
- [] اسم متغیرهای معنادار — حتی در کد سریع
- [] `edge case` را خودت بگو قبل اینکه بپرسند
- [] آخرش با یک مثال ذهنی تست کن و بلند بگو
- [] اگر گیر کردی: سکوت ممنوع — بلند فکر کن و راه ساده‌تر را پیشنهاد بده

→ **فصل بعد:** `System Design` برای فرانت‌کار — با پاسخ نمونه کامل.

فصل ۹ – System Design برای فرانت‌کار: با پاسخ نمونه کامل

🎯 هدف: «یک کامپوننت X را طراحی کن» یا «فرانت یک فروشگاه را طراحی کن» – ترسناک به نظر می‌رسد چون ساختار جواب ندارد. این فصل قالب ۵ مرحله‌ای + دو پاسخ نمونه کامل می‌دهد.

۹.۱ – قالب ۵ مرحله‌ای پاسخ (حفظ کن!)

۱. سوال بپرس – کاربران؟ داده؟ مقیاس؟ آفلاین؟ ۲-۳ (Requirements): شفاف‌سازی
۲. چیزهای داخل و خارج دامنه را صریح بگو (Scope): تعریف دامنه
۳. نقشه بلوک‌ها (صفحات، لایه‌ها، سرویس‌ها) – روی صفحه بکش معماری کلی:
۴. کش، پرفورمنس، ارور، دسترسی‌پذیری، state، عمق‌کاوی جزئیات
۵. «می‌کنم scale شد، این‌جوری X ها و توسعه آینده:» اگر Trade-off

💡 نکته طلایی: مصاحبه‌گر طرح درست واحد را نمی‌خواهد – می‌خواهد ببیند سوال می‌پرسی، trade-off می‌بینی و تصمیم‌ها را دلیل می‌آوری. سوال نپرسیدن و مستقیم کد نوشتن = پرچم قرمز بزرگ‌ترین.

۹.۲ – پاسخ نمونه ۱: «یک Autocomplete/جستجوی زنده طراحی کن»

مرحله ۱ – شفاف‌سازی (بلند بگو): - «نتایج از سرور می‌آید یا لیست لوکال؟» - (فرض: سرور) - «چند کاربر همزمان؟ سرعت API؟» - (فرض: API تا 200ms) - «موارد لازم: keyboard navigation؟ debounce؟ اخیراً جستجوها؟» - (فرض: بله همه)

مرحله ۲ – Scope: «شامل: dropdown، input، نتایج، highlight، keyboard nav، متن، loading و empty state. خارج از دامنه: صفحه نتیجه کامل، analytics.»

مرحله ۳ – معماری:

```
SearchBox (state - کلاینت)
├── Input (uncontrolled + debounce 300ms)
├── useSearch(query) hook
│   ├── AbortController - لغو درخواست قبلی
│   └── cache برای query Map ساده در
├── Results dropdown
│   ├── Skeleton لود
│   ├── keyboard: ↑↓ Enter Esc (roving focus)
│   └── aria: role="listbox" / aria-activedescendant
```

مرحله ۴ – عمق‌کاوی:

- **Race condition**: پاسخ حرف «a» دیرتر از «ab» برسد؟ → AbortController + چک رشته درخواست با آخرین query
- **پرفورمنس**: ۳۰۰ms debounce + حداقل ۲ حرف + AbortController؛ نتایج ≥ 10
- **State**: query → URL نه state (shareable) → use state؛ selected index → نتایج hook → از
- **a11y**: listbox/option، label برای focus، input مدیریت
- **ارور**: پیام «نتایج نیامد، دوباره تلاش کن» + retry — نه خاموشی
- **مرحله ۵ — Trade-off و آینده**: «برای داده لوکال، فیلتر سمت کلاینت با useMemo کافی بود — سرور برای داده بزرگ و خصوصی. آینده: highlight با `<mark>`، تاریخچه در SSR، localStorage نتیجه اولیه.»

۹.۳ — پاسخ نمونه ۲: «فرانت‌اند یک فروشگاه آنلاین را طراحی کن»

- **مرحله ۱ — شفاف‌سازی**: «تعداد محصول؟ ترافیک؟ سئو مهم است؟ (بله!) تیم بک‌اند چه API ای می‌دهد؟ ادمین جدا است؟»
- **مرحله ۲ — Scope**: داخل: لیست/جزئیات محصول، سبد، checkout، جستجو. خارج: پرداخت واقعی (با درگاه)، ادمین.
- **مرحله ۳ — معماری (این را بخش):**

```
Next.js App Router
├── / (لیست)           ISR + revalidateTag("products")
├── /product/[slug]    ISR + generateStaticParams + generateMetadata (سئو!)
├── /cart              Client Component (state/لوکال/Context) + localStorage sync
├── /checkout          Server Actions → درگاه → webhook
├── /api/webhooks/payment Route Handler (idempotent!)
├── auth               session cookie (httpOnly)
└── داده: Product → از DB با tags؛ تصاویر → next/image + remotePatterns
```

مرحله ۴ — عمق‌کاوی:

- **سئو**: صفحات محصول ISR + metadata + sitemap از DB — جستجوی گوگل حیات کسب‌وکار است
- **سبد خرید**: state کلاینتی (Context کوچک) + همگام با localStorage؛ بعد از لاگین با سرور sync؛ قیمت‌ها همیشه از سرور (امنیت!)
- **پرفورمنس**: لیست = ISR؛ تصاویر next/image با sizes؛ فیلترها با (shareable) searchParams؛ جزئیات کند → استریم ریویوها با Suspense
- **پرداخت**: Server Action → درگاه؛ webhook با idempotency key (دوباره زدن = دوبار شارژ نشود!)؛ خطا → پیام و retry
- **ارور و حالت‌ها**: empty state، out-of-stock، خطای درگاه با retry
- **a11y**: فرم‌ها با label، تمرکز مدیریت‌شده، کنتراست

مرحله ۵ – Trade-off و آینده: «ISR سرعت را می‌دهد ولی تازگی قیمت را ۶۰ ثانیه دیر – برای قیمت لحظه‌ای، fetch بدون کش در بخش قیمت جزئیات. اگر ترافیک جهانی شد: i18n + CDN region. اگر سبد سنگین شد: microservice سبد جدا.»

۹.۴ – سوالات دنباله‌دار (که بعد از طرحت می‌پرسند!)

سوال	الگوی جواب
«اگر ترافیک ۱۰× شد؟»	کش لایه‌ای (CDN/ISR) → read replica → صف‌ها → فقط بعداً microservice
«چطور تست می‌کنی؟»	واحد منطق + integration فرم‌ها + e2e مسیر خرید (Playwright)
«API کند شد چه می‌شود؟»	loading state، timeout + retry with backoff، skeleton، پیام صادقانه
«دسترسی‌پذیری؟»	aria، keyboard، semantic HTML، کنتراست – از ابتدا نه آخر پروژه
«کش را کی باطل می‌کنی؟»	webhook؛ mutation → revalidateTag از CMS/بک‌اند

جمع‌بندی فصل ✓

- قالب ۵ مرحله: شفاف‌سازی → Scope → معماری (بکش!) → عمق‌کاوی → Trade-off
- سوال پرسیدن اول = قوی‌ترین سیگنال senior بودن
- عمق‌کاوی = state/کش/پرفورمنس/ارور/a11y – همین پنج‌تا را همیشه پوشش بده
- هر تصمیم با دلیل و «اگر X شد، Y می‌کنم»

تمرین فصل ۹ 📝

1. دو پاسخ نمونه را **بلند** ارائه بده (ضبط کن) – هر کدام ۱۰-۱۵ دقیقه.
2. طراحی کن: «صفحه داشبورد ادمین با جدول داده بزرگ، فیلترها و Real-time» – با قالب ۵ مرحله.
3. طراحی کن: «سیستم کامنت‌گذاری با reply و لایک» – API، state، بهینه‌سازی.
4. برای پاسخ نمونه ۱، سه سوال شفاف‌سازی دیگر پیدا کن که نپرسیدم.

▼ راهنمای تمرین ۲ (اسکلت پاسخ)

1. شفاف‌سازی: چند ردیف؟ (۱۰هزار؟) فیلترها سمت سرور یا کلاینت؟ real-time چقدر جدی؟ صادرات اکسل؟
2. معماری: جدول سروری (Server Component با پارامترهای URL) + فیلترها با searchParams + virtualization برای ردیف‌ها + تعامل bulk با WebSocket/SSE + client island فقط برای بخش real-time واقعی (نوتیف).
3. عمق: pagination cursor (نه offset در این حجم) + index در DB بک‌اندی (پیشنهاد بده!) + skeleton + export در صف.
4. Trade-off: همه‌چیز کلاینت راحت ولی کند و سنگین؛ همه‌چیز سروری امن و سریع ولی تعامل کند – ترکیب جزیره‌ای بهترین است.

➔ **فصل بعد:** مصاحبه رفتاری – STAR با چهار جواب نمونه کامل.

فصل ۱۰ – مصاحبه رفتاری: STAR + چهار جواب نمونه کامل

🎯 هدف: «Tell me about a time...» – سوالاتی که جوابشان تجربه‌های واقعی توست. یاد می‌گیری با قالب STAR داستان‌های خودت را در چهار داستان همه‌کاره ذخیره کنی که به ده‌ها سوال جواب می‌دهند.

۱۰.۱ – قالب STAR (و ترجمه‌اش به فارسی)

S – Situation: صحنه در دو جمله (کجا، چه خبر بود)
T – Task: مسئولیت مشخص تو چه بود
A – Action: «من» چه کردم (بگو «ما» نه – تو بگو!)
R – Result: نتیجه با عدد/اثر + درس گرفته

قواعد: - داستان‌ها کوتاه: ۹۰-۱۲۰ ثانیه - **Action بخش بزرگ‌ترین است** (نصف زمان) - نتیجه با عدد: «زمان لود نصف شد»، «سه sprint صرفه‌جویی» - حتی در شکست: درس گرفته + تغییر رفتار بعدی را بگو

چهار داستان همه‌کاره بساز (از پروژه‌های واقعی خودت!) که این سوال‌ها را پوشش دهند: ۱. یک مشکل فنی سخت ۲. یک تعارض/اختلاف نظر ۳. یک شکست/اشتباه ۴. یک موفقیت/یادگیری سریع

۱۰.۲ – جواب نمونه ۱: «Tell me about a technical challenge»

سوال: «سخت‌ترین چالش فنی که حل کردی؟»

جواب نمونه (شما را با پروژه‌های خودت جایگزین کن):

S: در فروشگاه‌ای که ساختم، صفحه سید خرید با ورود ۵۰ آیتم به بالا به شدت کند می‌شد و تایپ در فیلد کد تخفیف لگ داشت.

T: بک‌اند API مالکیت بهینه‌سازی همان صفحه با من بود - بدون تغییر.

A: پیدا کردم هر تایپ، کل لیست ۵۰ آیتم React DevTools Profiler با می‌کرد. سه کار کردم: فیلد تخفیف را به کامپوننت جدا re-render را useMemo کردم، و محاسبه جمع کل را به memo منتقل کردم، ردیف‌ها را با وابستگی درست بردام.

R: Lighthouse و (۶۰fps رفتار) رسید ۱۶ms زیر ۴۰۰ms تایپ از ۹۲ - از ۶۵ به ۹۲. درسی که گرفتم: اول پروفایل، بعد بهینه interaction بهینه‌سازی کورکورانه فقط پیچیدگی می‌سازد.

چرا این جواب قوی است: ابزار مشخص (Profiler)، تصمیم‌های مشخص، نتیجه عددی، درس گرفته - و همگی قابل راستی‌آزمایی.

۱۰.۳ – جواب نمونه ۲: «یک اختلاف نظر با هم‌تیمی»

سوال: «Tell me about a disagreement with a teammate».

جواب نمونه:

- S: offset همکار یک‌اند می‌خواست لیست محصولات را با X در پروژه لازم داشتیم cursor فید و سئو UX بدهد؛ من برای pagination
- T: تصمیم مشترک بود و دی‌لاین فشرده داشتیم – بحث تبدیل به بن‌بست شده بود در چت.
- A: به‌جای ادامه چت، یک جلسه ۱۵ دقیقه‌ای گذاشتیم و اول هر دو گزینه را در صفحه ۵۰ با ۱۰ هزار رکورد چه کوئری‌ای offset: با اعداد نوشتیم چطور سئوی صفحه‌بندی را پیچیده می‌کند. بعد روی cursor، می‌سازد. برای پنل ادمین offset، اصلی feed برای cursor: داده‌ها تصمیم گرفتیم
- R: هر دو طرف خواسته‌اش را گرفت و بحث ۳۰ دقیقه‌ای به تصمیم ۱۵ دقیقه‌ای تبدیل شد. درس: اختلاف فنی را با داده حل کن، نه با مقاومت؛ و تصمیم با عدد»، دوستی‌ها را حفظ می‌کند».

چرا قوی است: ضدالگو را هم نشان می‌دهد (چت کشدار بد بود) + فرایند حل (داده) + نتیجه دوطرفه + درس.

۱۰.۴ – جواب نمونه ۳: «یک اشتباه/شکست» (سوال ترسناک!)

سوال: «Tell me about a time you failed».

جواب نمونه:

- S: یک تغییر اسکیمای main اوایل کارم، بدون هماهنگی مستقیم روی شاخه دیتابیس را دیپلوی کردم که فرم ثبت‌نام را برای ۲۰ دقیقه شکست
- T: بعد از کشف، حل مشکل و شفاف‌سازی با تیم با من بود.
- A: کردم و در کانال تیم اعلام کردم – قبل از اینکه rollback بلافاصله کوچک نوشتم و یک گیت به فرایند اضافه postmortem کسی بفهمد. بعد می‌شوند review و توسط نفر دوم PR ها فقط با migration: شد
- R: از آن روز چنین حادثه‌ای تکرار نشد و همان گیت بعداً برای تیم‌های دیگر هم الگو شد. اشتباه من گوشه‌گیری بود؛ یاد گرفتم خبر بد را سریع و خودت بگو – اعتماد تیم از عذرخواهی مهم‌تر است

چرا قوی است: اشتباه واقعی (نه «perfectionist بودم»!)، مسئولیت بی‌تعارف، اصلاح سیستمی (نه فقط پوز)، درس ماندگار. مصاحبه‌گر می‌خواهد ببیند با شکست بالغ برخورد می‌کنی.

۱۰.۵ – جواب نمونه ۴: «یادگیری سریع زیر فشار»

سوال: «Tell me about a time you had to learn something quickly».

جواب نمونه:

- S: باشد - درحالی‌که CMS وسط پروژه، مشتری خواست بخش اخبار سایت با دستی ساخته بودم API من تا آن روز فقط.
- T: دو هفته تا ددلاین؛ لایه محتوا با من بود.
- A: بک‌اند انتخاب کردم: پنل آماده برای headless وردپرس را به عنوان POC برای من. سه روز مستندات رسمی، بعد یک REST/GraphQL + مشتری. برای مشتری گذاشتم preview و caching کوچک، و باقی زمان را روی سر موعد تحویل شد؛ مشتری الان خودش محتوا می‌گذارد و ما صفحه‌ها تازه نگه می‌داریم. یاد گرفتم ابزار را با نیاز مشتری ISR را با قبل از تعهد، ریسک را صفر POC انتخاب کنم، نه با علاقه شخصی - و می‌کند.

(این دقیقاً همان داستان دوره وردپرس Headless توسست - تجربه واقعی داشته باش، جواب واقعی می‌شود!)

۱۰.۶ - سوالات رفتاری پرتکرار (به داستان‌های map کن!)

سوال	داستان مناسب
سخت‌ترین چالش فنی؟	داستان ۱
اختلاف نظر؟	داستان ۲
شکست؟	داستان ۳
زیر فشار؟	داستان ۳ یا ۴
چیزی که سریع یاد گرفتی؟	داستان ۴
اولویت‌بندی چند کار؟	داستان ۱ (با بخش اولویت)
feedback سخت گرفتی؟	داستان ۳ (اشتباه + پذیرش)
چیزی که بهش افتخار می‌کنی؟	داستان ۱

جمع‌بندی فصل ✓

- STAR: صحنه ۲ جمله، مسئولیت، **Action نصف زمان**، نتیجه با عدد + درس
- چهار داستان همه‌کاره از پروژه‌های واقعی خودت بساز و بلند تمرین کن
- سوال شکست: اشتباه واقعی + اصلاح سیستمی + درس - نه تواضع کاذب
- «ما» نه - «من» بگو؛ و همیشه یک جمله «درس گرفته‌شده» آخر داستان

تمرین فصل ۱۰

1. چهار داستان خودت را با STAR بنویس (از پروژه‌های واقعی) – هر کدام زیر ۱۲۰ کلمه.
2. هر چهار تا را ضبط کن و بشنو – کجاها «ما» گفتی؟ کجا عدد نداری؟
3. از یک دوست بخواه سه سوال رفتاری تصادفی بپرسد – با داستان‌هایت جواب بده.
4. سوال «ضعف شما چیست؟» را جواب بده (فرمول: ضعف واقعی کوچک + اقدام مشخص برای بهبود – نه «perfectionist هستم»!).
5. «چرا می‌خواهی با ما کار کنی؟» را برای یک شرکت واقعی بنویس (از فصل ۳: تحقیق محصول!).

▼ نمونه جواب تمرین ۴

«در داکيومنت‌نویسی و ارتباط پیش‌دستانه ضعیف بودم – کار را خوب انجام می‌دادم ولی به‌موقع اطلاع نمی‌دادم و همکارها غافلگیر می‌شدند. بعد از یک feedback سخت، دو کار کردم: هر روز در standup وضعیت دقیق می‌گویم، و برای کارهای بیش از دو روز، وسط راه هم آپدیت می‌دهم. الان همکارها می‌گویند وضعیت پروژه‌ها از من شفاف است – چون سنجیده‌اش می‌کنم.»

(فرمول: ضعف واقعی غیرمرگبار + اقدام مشخص + شاهد تغییر.)

➔ **فصل بعد:** سوالاتی که تو می‌پرسی + red flag های شرکت + گفت‌وگوی حقوق.

فصل ۱۱ – سوالاتی که تو می‌پرسی + Red Flags + گفت‌وگوی حقوق

🎯 هدف: مرحله «سوال داری؟» فرصت طلایی است – نه تشریفات. سوالات تو = سیگنال حرفه‌ای بودن + داده برای تصمیم خودت. و گفت‌وگوی حقوق را هم بدون استرس یاد بگیر.

۱۱.۱ – چرا سوال پرسیدن امتیاز است؟

- نشان می‌دهد فکر می‌کنی، فقط دنبال جاب نیستی
- داده می‌دهد برای تصمیم تو (آیا این شرکت برای من خوب است؟)
- سوال نپرسیدن = سیگنال بی‌علاقگی – بله واقعاً برعکس تصور!

۱۱.۲ – سوالات درجه‌یک (به ترتیب ارزش)

درباره کار (به hiring manager):

★ «موفقیت در سه ماه اول این نقش چه شکلی است؟»
(پرامتیازترین سوال – نشان می‌دهد نتیجه‌محور فکر می‌کنی)

«بزرگ‌ترین چالش فنی تیم الان چیست؟»
«و دیپلوی شما چطور است؟ code review فرایند»
«شود؟ (بک‌لاگ/دیتا) build چطور تصمیم می‌گیرید چه چیزی»
«تکنولوژی‌ها چقدر الان با چیزی که در آگهی است یکی است؟»

درباره تیم و رشد:

«تیم چند نفر است و ساختارش چطور است؟»
«برای این نقش چطور تعریف می‌شود؟ growth انتظار»
«چطور انجام می‌شود؟ evaluation بازخورد و»

درباره روزمره:

«یک روز معمولی این نقش چطور می‌گذرد؟»
«ها چقدر است؟ sync ها و meeting»
«دورکاری چطور هندل می‌شود؟»

۱۱.۳ – Red Flags: نشانه‌های خطر در مصاحبه

پرچم قرمز	یعنی معمولاً
«ما مثل یک خانواده‌ایم»	مرز کار/زندگی محو – اضافه‌کاری «خویشاوندانه» انتظار می‌رود
«حقوق به تلاشت بستگی دارد»	ساختار پرداخت مبهم = مذاکره‌ناپذیر
مصاحبه‌گرها درباره هم تیمی بد می‌گویند	فرهنگ سمی – تو هم هدف بعدی خواهی بود
«همه‌چیز از صفر ساخته می‌شود» + ددلاین ۲ ماه	chaos factory – بدون طراحی، با اضافه‌کاری
از سوال «چرا نفر قبلی رفت؟» فرار می‌کنند	داستان پنهانی هست
پروسه مصاحبه بی‌نظم (دیر می‌آیند، فراموش می‌کنند)	نمونه‌ای از فرهنگ روزمره‌شان
«اگه عاشق کار باشی دیگه حس ۹-۵ نداری»	استثمار پوشیده با اشتیاق

💡 به رفتارشان در خود مصاحبه هم دقت کن: دیر آمدند؟ بدون خواندن رزومهات سوال پرسیدند؟ به سوالات تو بی‌حوصله جواب دادند؟ – اینها preview ی روزهای کاری توست.

۱۱.۴ – گفت‌وگوی حقوق: بدون استرس، حرفه‌ای

قوانین چهارگانه:

1. اول محدوده را از آن‌ها بگیر: «رنج پرداختی این نقش چقدر است؟» – قبل از گفتن عددت
2. اگر مجبوری عدد بگو، بازه بگو: «با توجه به بازار و سابقه‌ام، بازه‌ی X تا Y انتظار دارم»
3. عددت را تحقیق کرده باش: آگهی‌های مشابه، همکاران، سطوح – حدس زن
4. منفی نگو، گزینه باز بگذار: «فعلاً آماده نیستم» به جای «نه!»

دیالوگ نمونه (وقتی حقوق فعلی را می‌پرسند):

آن‌ها: الان چقدر می‌گیری؟
 - تو: ترجیح می‌دم تمرکز روی ارزشی باشه که می‌تونم اضافه کنم.
 هست Y تا X رنج انتظار من با توجه به بازار و تجربه‌ام.
 با بودجه این نقش چطور می‌خونه؟

(قانون: حقوق فعلی محرمانه است و لازم نیست گفته شود – مودبانه منحرف کن به بازه انتظار.)

وقتی آفر آمد و کم است:

ممنون از پیشنهاد – از فرصت واقعاً هیجان‌زده‌ام. با توجه به تجربه‌ام،
 بود Y و رنج بازار برای این نقش، انتظارم نزدیک‌تر به [X] در
 «آیا روی پایه حقوق انعطاف هست، یا بخش دیگری از پکیج؟»

(مودبانه، داده‌محور، باز – و «بخش دیگری از پکیج» یعنی bonus/روزهای دورکاری/آموزش هم قابل مذاکره است.)

⚠ هرگز: دروغ گفتن درباره آفر رقیب (چک می‌شود/اعتماد می‌سوزد)، پذیرش فوری زیر فشار («بگذار ۲۴ ساعت فکر کنم» کاملاً استاندارد است!)، یا عصبانی شدن – مذاکره = گفت‌وگوی تجاری، نه جنگ.

۱۱.۵ – سوالاتی که نباید اول بپرسی ❌

- مرخصی و تعطیلات (اولین سوال = «دنبال راحتی‌ام») – بعد از آفر بپرس
- «آیا اضافه‌کاری داریم؟» مستقیم – به‌جایش: «یک هفته شلوغ چطور می‌گذرد؟»
- جزئیات حقوقی ریز قبل از علاقه متقابل

جمع‌بندی فصل ✅

- سوال پرسیدن = امتیاز؛ سوال ستاره: «موفقیت سه ماه اول؟»
- red flags: «خانواده»، حقوق مبهم، بی‌نظمی فرایند، بدگویی درباره تیم
- حقوق: اول رنج آن‌ها، بازه تحقیق‌شده، مذاکره مودبانه و داده‌محور
- آفر کم: ممنون + بازه انتظار + «بخش دیگری از پکیج؟» + ۲۴ ساعت فکر

تمرین فصل ۱۱ 📄

1. پنج سوال این فصل را انتخاب کن و برای شرکت هدف‌ات شخصی‌سازی کن.
2. تمرین دیالوگ حقوق با یک دوست: او recruiter باشد و سه بار فشار بیاورد («عدد دقیق بگو») – تو سه بار مؤدبانه به بازه برگرد.
3. سه red flag را که در تحقیق یک شرکت واقعی (گلس‌دور/نظرات کارکنان) پیدا کردی بنویس.
4. جمله مذاکره را برای آفر فرضی ۳۰ میلیون (انتظار ۴۵) بنویس.

▼ نمونه جواب تمرین ۴

«از پیشنهاد و اعتمادتون واقعاً ممنونم – علاقه‌ام به تیم و پروژه جدیه. با توجه به تجربه‌ی من در Next.js و پروژه‌های دپلوی‌شده‌ام، و رنج بازار برای این سطح، انتظارم نزدیک‌تر به بازه ۴۰ تا ۴۵ بود. آیا روی پایه حقوق انعطافی هست؟ یا می‌شه روی پاداش/آموزش/روزهای دورکاری هم فکر کرد؟ – برای تصمیم نهایی، ۲۴ ساعت فرصت می‌خوام.»

➔ فصل بعد: بعد از مصاحبه – follow-up، رد شدن، و ۹۰ روز اول.

فصل ۱۲ – بعد از مصاحبه: follow-up، رد شدن، مذاکره و ۹۰ روز اول

🎯 هدف: فرایند با بیرون آمدن از اتاق مصاحبه تمام نمی‌شود – و مهم‌تر: رد شدن‌ها را چطور پردازش کنی که اعتماد به نفست نخورد. آخرش هم ۹۰ روز اول شغل جدید.

۱۲.۱ – Follow-up: ایمیل ۲۴ ساعته

interview [عنوان موقعیت] – Thank you موضوع

سلام [اسم مصاحبه‌گر]

بحث برایم جالب بود [مشخص X] ممنون از گفت‌وگوی امروز – بخش مخصوصاً [یک جزئیات واقعی که گفت]

ضمناً در مورد [سوالی که پرسید و جواب کامل ندادی] فکر کردم. گفتم شاید مفید باشد – [دو خط جواب تکمیلی]

منتظر قدم بعدی هستم.
[امضا]

چرا: (۱) ادب حرفه‌ای، (۲) فرصت دوم برای سوال جواب‌نداده (امتیاز برگرداندن!)، (۳) یادآوری تو در ذهن او. کوتاه بماند – سه پاراگراف.

⚠️ spam نکن: یک follow-up + اگر ۵-۷ روز بی‌جواب بود یک پیگیری کوتاه – بیشتر نه. بی‌جوابی گاهی یعنی «فرایند کند است» نه «رد شدی».

۱۲.۲ – رد شدن: پردازش سالم

واقعیت آماری: رد شدن معمول است – حتی برای کاندیدای عالی. علل رد شدن فقط مهارت نیست: بودجه بسته شد، نفر داخلی را ترجیح دادند، تغییر اولویت شرکت، سلیقه‌ی team fit.

پروتکل ۴۸ ساعته بعد از رد شدن:

۱. اجازه بده ناراحت باشی (یک روز) – طبیعی است
۲. درخواست feedback: «ممنون از فرصت – اگر ممکن است بفرمایید چه بخشی را می‌توانستم بهتر انجام دهم؟» (بعضی‌ها جواب می‌دهند – طلاست)
۳. داده بردار: در جدول اپلای‌ها بنویس: چه مرحله؟ چه سوالی گیر کردم؟

4. یک اصلاح مشخص انتخاب کن – نه بازنویسی کل شخصیت: «فیل ۸ را بهتر تمرین می‌کنم»

5. همان هفته، اپلای بعدی – بهترین پادزهر رد شدن، مومن‌توم است

🔑 بازچینش: مصاحبه‌های رد شده = تمرین رایگان با شرایط واقعی. کسی که ۱۰ مصاحبه داده و رد شده، در مصاحبه یازدهم از کسی که اولین‌اش است جلوتر است – حتی با دانش برابر.

و اگر feedback نیامد: خودت دیتا جمع کن – سوال‌ها را در فصل ۱۲ یادداشت کرده‌ای (فصل ۱۳!). الگوها را ببین: همیشه در live coding گیر می‌کنی؟ → فصل ۸. همیشه behavioral ضعف؟ → فصل ۱۰.

۱۲.۳ – چند آفر: مقایسه حرفه‌ای

معیار	سوال دقیق از خودت
پایه حقوق	با بازار و تجربه‌ام منصفانه است؟
رشد	یاد می‌گیرم؟ mentor هست؟ تکنولوژی مُدرن؟
تیم و لید	در مصاحبه‌ها چطور برخورد کردند؟
محصول	باور دارم؟ (۳ سال از عمرت!)
پایداری شرکت	درآمدش از کجاست؟ (حتی قبل حقوق!)
دورکاری/ساعت	واقعاً چقدر؟

⚠️ حقوق بالاتر با تیم بدتر = توافق معاملاتی‌ای که سه ماه بعد پشیمانی می‌خرد. رشد دو ساله از حقوق ۲۰٪ بیشتر ارزش‌ساز است (در بازار کار بعدی چند برابر برمی‌گردد).

۱۲.۴ – استعفا از شرکت فعلی (اگر شاغلی)

- نامه استعفا کوتاه و بی‌تنش: تاریخ آخرین روز + تشکر – بدون انتقاد
- notice period را رعایت کن (قرارداد/عرف: ۱-۲ ماه) – دنیای توسعه کوچک است، شهرت تو ماندگار است
- تحویل کار: مستندسازی + handoff تمیز (همین حرفه‌ای‌بودن، بعداً reference خوب می‌شود)
- به همکاران نگو قبل از اینکه رسمی شود

۱۲.۵ – ۹۰ روز اول شغل جدید (برنامه سه ماهه)

دوره	تمرکز	کارهای مشخص
روز ۱-۳۰	فهمیدن	کد را بخوان (نه تغییر بده)، setup کامل، با تک تک تیم ۱:۱ گفتگو («کجای کد خطرناک است؟»)، اولین PR کوچک
روز ۳۱-۶۰	کمک کردن	تیکت های واقعی، ownership یک بخش، سوال های «چرا این طوریه؟» را مستند کن
روز ۶۱-۹۰	پیشنهاد	یک بهبود مشخص پیشنهاد بده (با داده)، نقطه ی تخصصی ات را معرفی کن، بازخورد بگیر

💡 جمله ای که در هفته اول معجزه می کند: «برای من عجیب نیست که این جا متفاوت باشد – فقط بگویند استاندارد شما چیست.» – بدون مقایسه با شرکت قبلی (بزرگ ترین خطای کاندیداهای باتجربه!).

جمع بندی فصل ✓

- ۲۴ follow-up ساعته: تشکر + تکمیل جواب سوال + کوتاه
- رد شدن: پروتکل ۴۸ ساعته – feedback بگیر، یک اصلاح مشخص، همان هفته بعدی
- چند آفر: رشد و تیم $\leq$ حقوق؛ سه سال عمرت را می فروشی
- استعفا: notice رعایت + handoff تمیز = reference ماندگار
- ۹۰ روز: بفهم → کمک کن → پیشنهاد بده؛ «استاندارد شما چیست؟»

تمرین فصل ۱۲ 📝

1. قالب follow-up email را برای آخرین مصاحبه واقعیات بنویس (یا یک مصاحبه شبیه سازی).
2. رد شدن های قبلیات را مرور کن: الگوی مشترک چی بود؟ یک اصلاح مشخص برای بعدی انتخاب کن.
3. جدول مقایسه دو آفر فرضی را پر کن (حقوق/رشد/تیم/محصول/دورکاری) و تصمیمت را با دلیل بنویس.
4. برنامه ۳۰ روز اولت را برای شغل رویاییات بنویس (۱۰ خط).

▼ نمونه جواب تمرین ۲ (الگوی واقعی)

«دو رد شدن اخیرم هر دو در مرحله live coding بود – هر دو بار در بخش state management تردید داشتم و بلند فکر نکردم. اصلاح مشخص: هفته آینده هر روز یک تمرین فصل ۸ با ضبط صدا، و قالب ۵ قدمی Thinking Out Loud را حفظ می کنم. تمرین بعدی هم صرفاً algوریتمی نیست – دقیقاً همین قالب را تمرین می کنم.»

[→](#) **فصل آخر:** چیت‌شیت‌ها + گلاساری + نقشه راه.

فصل ۱۳ – چیت‌شیت نهایی، گلاساری و نقشه راه

🎯 هدف: همه‌چیز که روز مصاحبه لازم داری در یک فصل – چک‌لیست‌ها، جواب‌های آماده، گلاساری و نقشه راه.

۱۳.۱ – چک‌لیست هفته قبل از مصاحبه

- [] شرکت را تحقیق کن: محصول (واقعاً استفاده کن!)، اخبار، team، glassdoor-like، نظرات
- [] رزومه‌ای که فرستادی را دوباره بخوان – سوالات از دل آن می‌آیند!
- [] سه پروژه پرچم‌دارت: What/Why/How/چالش – آماده تعریف
- [] فصل ۵-۷ (سوالات فنی) را بلند مرور کن
- [] ۴ جواب STAR را مرور کن (فصل ۱۰)
- [] ۳ سوال برای پرسیدن آماده کن (فصل ۱۱)
- [] حقوق: بازار را چک کن، بازه‌ات را بنویس
- [] تمرین: یک live coding با تایمر + یک مصاحبه شبیه‌سازی

۱۳.۲ – چک‌لیست روز مصاحبه

- [] ۱ ساعت قبل: تجهیزات (آنلاین)/مسیر (حضور)، چیت‌شیت نیم‌صفحه، آب
- [] ۲۰ دقیقه قبل: warm-up (کد یا داکيومنت)
- [] ۵ دقیقه قبل: تنفس ۴-۷-۸ + جمله لنگر
- [] حین مصاحبه: Thinking Out Loud (۵ قدم) + سوال بپرس + «نمی‌دانم» حرفه‌ای
- [] آخر: ۲ سوال بپرس + «قدم بعدی فرایند چیست؟»
- [] همان روز: یادداشت سوال‌ها + ایمیل follow-up

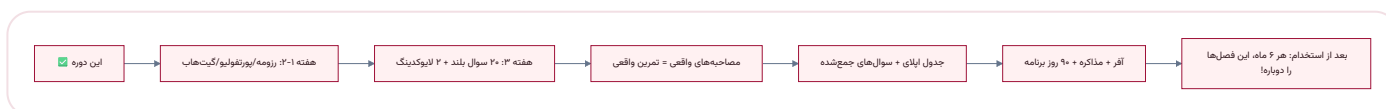
۱۳.۳ – جواب‌های آماده (خلاصه فصل‌های ۱۰-۱۲)

موقعیت	جمله کلیدی
«خودت را معرفی کن»	مسیر + تخصص + یک دستاورد عددی + چرا اینجا (۲ دقیقه)
«نمی‌دانی؟»	«X را بلد نیستم – ولی Y را می‌دانم که نزدیک است و برای X این‌طور می‌فهمم...»
«چرا ما؟»	یک واقعیت از محصولشان + تطابق با مهارت
«شکست؟»	اشتباه واقعی + اصلاح سیستمی + درس (فصل ۱۰)
«حقوق؟»	«رنج شما؟» → بازه تحقیق‌شده → «بخش دیگری از پکیج؟»
«سوال داری؟»	«موفقیت سه ماه اول؟» + چالش فنی تیم + ساختار

۱۳.۴ – گلاساری فرایند استخدام

اصطلاح	یعنی
JD / job posting	شرح شغل
screening	مکالمه غربال اولیه
technical / live coding / system design	مراحل فنی
behavioral + STAR	رفتاری + قالب جواب
hiring manager / recruiter	مدیر استخدام / جذب کار
take-home	پروژه خانه
whiteboard / pair programming	تخته ای / جفتی
offer / negotiate	پیشنهاد / مذاکره
base / bonus / equity	پایه / پاداش / سهام
notice period / probation	اطلاع قبلی / آزمایشی
onboarding / onboarding buddy	ورود / راهنمای روز اول
follow-up / rejection / counter-offer	پیگیری / رد / پیشنهاد متقابل
red flag	پرچم قرمز
imposter syndrome	سندروم دغل کار
thinking out loud	بلند فکر کردن
STAR	Situation-Task-Action-Result
YoE	years of experience

۱۳.۵ – نقشه راه پس از این دوره



سه توصیه پایانی:

1. مصاحبه = مهارت، نه بخت: مثل هر مهارتی با تکرار ساخته می‌شود – مصاحبه‌ی شبیه‌سازی‌شده با دوست، مؤثرتر از ۱۰ ساعت خواندن سوال است.

2. دیتابیس سوال‌ها = گنج تو: هر مصاحبه (حتی رد شده) سوال‌هایش را ثبت کن – بعد از ۵ مصاحبه، لیست سوالات شخصی‌ات دقیق‌تر از هر سایت است.

3. پس از استخدام هم این دوره زنده است: هر بار خواستی شغل عوض کنی، همین چرخه (رزومه → قیف → مراحل → مذاکره) دوباره اجرا می‌شود – فقط هر بار مسلط‌تر.

پایان دوره: تو حالا رزومه‌ی بهینه، قیف اپلای، جواب‌های ۸۹ سوالی، قالب STAR، سوالات خودت و برنامه ۹۰ روز داری. ترس مصاحبه از «ناشناخته» می‌آید – و تو دیگر هیچ‌چیز ناشناخته‌ای نداری. برو و آن گفت‌وگوی دوطرفه را برگزار کن! 🙌
(و در ادامه، فصل‌های ۱۴ و ۱۵ – ۳۵ سوال تکمیلی HTML/CSS و تله‌های JS از رپیوهای مرجع – گرم‌کننده‌ی هر مصاحبه‌ات خواهند بود.)

فصل ۱۴ – ۲۰ سوال HTML/CSS با جواب کامل

🎯 هدف: بزرگ‌ترین شکاف مصاحبه‌های فرانت بعد از JS: سوالات HTML و CSS. این فصل از سوالات واقعی **Front End Interview Handbook** (مجموعه‌ی معروف h5bp) استخراج شده و هیچ هم‌پوشانی با فصل‌های ۵-۷ ندارد. هر سوال: جواب کوتاه + نکته عمیق.

💡 سوالات HTML/CSS معمولاً «گرم‌کننده» اول مصاحبه‌اند – جواب سریع و مطمئن در این بخش، لحن کل مصاحبه را تعیین می‌کند.

HTML – ۱۴.۱

Q1: Semantic HTML چیست و چرا مهم است؟

جواب: استفاده از تگ‌های معنادار (`header` ، `nav` ، `main` ، `article` ، `section` ، `footer` ، `figure`) به جای `div`-`soup`؛ مرورگر و اسکرین‌ریدر «معنا» را می‌فهمند، نه فقط شکل.

عمیق‌تر: سه پرنده دارد: دسترسی‌پذیری (پیمایش با landmark ها)، سئو (خزنده ساختار را می‌فهمد) و خودت (خوانایی کد). تست ساده: صفحه بدون CSS هنوز باید قابل فهم باشد.

Q2: DOCTYPE چه می‌کند؟

جواب: به مرورگر اعلام می‌کند سند را در **standards mode** رندر کند. بدون آن (یا با doctype ناقص قدیمی) مرورگر به **quirks mode** می‌رود: box model قدیمی و رفتارهای ناسازگار بین مرورگرها.

عمیق‌تر: `<!DOCTYPE html>` همین و فقط همین. برای چک کردن mode فعال: `document.compatMode` → `"CSS1Compat"` یعنی standards.

Q3: فرق `<script async>` ، `<script>` و `<script defer>` ؟

جواب: هر سه داندلودشان همزمان با پارس HTML شروع می‌شود؛ فرق در اجرا است:

حالت	کی اجرا می‌شود؟	ترتیب
<code><script></code>	فوری بعد از داندلود — پارس HTML را بلاک می‌کند	به‌ترتیب سند
<code>async</code>	هر وقت داندلود تمام شد	نامعلوم!
<code>defer</code>	بعد از پارس کامل، درست قبل DOMContentLoaded	به‌ترتیب سند

عمیق‌تر: اسکریپت مستقل (analytics) → `async`؛ اسکریپت‌های وابسته به DOM و به‌هم → ES Module. `defer` ها به‌طور پیش‌فرض رفتار `defer` دارند — و دیگر لازم نیست `script` را آخر `body` بگذاری (سوال بعدی).

Q4: چرا CSS را در `head` و `script` را (قدیم‌ها) آخر `body` می‌گذاشتند؟

جواب: CSS در `head` تا اولین رندر استایل کامل داشته باشد (وگرنه **FOUC** — فلش محتوا بدون استایل)؛ `script` پایین صفحه تا پارس HTML بلاک نشود و کاربر زودتر محتوا ببیند.

عمیق‌تر: CSS رندر-بلاکینگ است (CSSOM باید آماده شود) و JS پارس-بلاکینگ. جایگزین مدرن `script` پایین صفحه همان `defer` در `head` است — و در Next.js کل این چیدمان را فریم‌ورک مدیریت می‌کند (فصل ۷). ایده‌ی «بخش‌بخش فرستادن صفحه» همان است که امروز streaming می‌نامیم (فصل ۷ Q6).

Q5: attribute های `data-*` به چه دردی می‌خورند؟

جواب: جای استاندارد چسباندن داده‌ی سفارشی به المان: `data-product-id="42"` — خواندن با `el.dataset.productId` (خط‌ها camelCase می‌شوند).

عمیق‌تر: کاربردهای واقعی: سلکتور تست (`data-testid`) در state، (Testing Library کوچک UI، و event delegation (فصل بعد Q1). تله: داده‌ای که به React باید برسد در state/props است — `data-*` برای کد خارج از ری‌اکت است.

Q6: صفحه‌ی چندزبانه چطور می‌سازی؟

جواب: `lang` درست روی `<html>` (مثلاً `dir="rtl"`، `fa` برای راست‌به‌چپ، `hreflang` روی `<link>` برای معرفی نسخه‌های زبانی به موتور جستجو، و آدرس مجزا برای هر زبان (مسیر یا زیردامنه).

عمیق‌تر: نکته‌های طراحی چندزبانه: طول متن‌ها فرق دارد (آلمانی ~۳۰٪ بلندتر) — layout را با flex/grid انعطاف‌پذیر بگیر، نه عرض ثابت؛ فونت و line-height مخصوص هر اسکریپت؛ تاریخ/عدد با Intl. برای فارسی: فونت وزیرمتن + دقت در `dir` که متن مخلوط (اعداد/انگلیسی داخل فارسی) به‌هم نریزد.

Q7: `srcset` در `<img>` چه می‌کند و مرورگر چطور انتخاب می‌کند؟

جواب: چند فایل با عرض‌های مختلف پیشنهاد می‌دهی و با `sizes` اعلام می‌کنی تصویر در viewport من چند پیکسل جا می‌گیرد؛ مرورگر با DPR و عرض واقعی، بهترین گزینه را خودش برمی‌دارد:

```

```

عمیق‌تر: `width/height` صریح = CLS صفر؛ `loading="lazy"` = داندود وقتی نزدیک viewport. در Next همه‌ی این‌ها را `next/image` خودکار می‌کند (فصل ۷ Q13) – ولی در پروژه‌های بدون فریم‌ورک، `srcset` سوال رایج است.

CSS – ۱۴.۲

Q8: box model را توضیح بده؛ * { box-sizing: border-box } چه می‌کند؟

جواب: هر المان `content + padding + border + margin` پیش‌فرض `content-box` است: `width` فقط `content` را می‌شمارد؛ با `border-box`، عرض شامل `padding` و `border` هم می‌شود – محاسبه‌ی layout انسانی می‌شود. **عمیق‌تر:** تقریباً همه‌ی `reset` های مدرن (و Bootstrap) با `border-box` شروع می‌کنند. فرق `reset` (همه‌چیز را صفر می‌کند) و `normalize` (پیش‌فرض‌های مفید را حفظ و فقط ناسازگاری‌ها را یکسان می‌کند) را هم یک جمله بلد باش.

Q9: Specificity چگونه محاسبه می‌شود؟ !important کی؟

جواب: وزن سه‌رقمی: `(id, class/attribute/pseudo-class, element)` – مثلاً `#nav .link` با `(1,1,0)` از `a:hover` با `(0,1,1)` می‌چربد. مساوی بودن؟ ترتیب سورس: آخرین می‌برد. `!important` از همه بالاتر است. **عمیق‌تر:** مرورگر سلکتور را از راست به چپ می‌کند (اول آخرین بخش را چک می‌کند – ارزان‌ترین). ترفند مدرن: specificity `where(#id, .class)` صفر می‌دهد – برای `reset` های ایمن. جواب مصاحبه‌ای `!important`: «برای غلبه بر استایل ابزار بیرونی (مثل `override` کردن بوت‌استرپ) رایج است، ولی مسیر درست اول specificity درست است – `important` آخرین راه‌حل.»

Q10: block, inline و inline-block چه فرقی دارند؟

جواب: `block` تمام عرض خط را می‌گیرد و `width/height` می‌پذیرد؛ `inline` در جریان متن می‌ماند و `width/height` نمی‌پذیرد؛ `inline-block` در جریان متن است ولی ابعاد می‌پذیرد – ابزار کلاسیک `badge` و دکمه‌ی کوچک. **عمیق‌تر:** تله‌ی کلاسیک `inline: vertical padding` پس‌زمینه را رنگ می‌کند ولی layout نمی‌زند (روی خط مجاور می‌ریزد). مدرن: با `flex/grid` کمتر به `inline-block` نیاز داری؛ `inline-flex` برای آیتم درون‌متنی فلکسی.

Q11: مقادیر position را توضیح بده؟

جواب: `static` (پیش فرض)؛ `relative` از جای عادی جابجا می شود و **مرجع absolute** فرزندانش است؛ `absolute` نسبت به نزدیک ترین جد `non-static`؛ `fixed` نسبت به viewport؛ `sticky` ترکیب `relative+fixed` – تا رسیدن به آستانه عادی، بعد چسبیده.

عمیق تر: تله های `fixed` جدی که `transform/filter/perspective` دارد مرجع آن می شود و «`fixed` بودن» می شکند. `sticky` به `overflow: hidden` در جد حساس است (اسکرول کانتینر می شود). کاربرد `sticky`: هدر جدول و sidebar.

Q12: Flexbox یا Grid؟

جواب: `flex` برای یک بعدی (toolbar، ردیف کارت، بین فاصله ها)؛ `grid` برای دوبعدی (layout صفحه، گالری با ستون های منظم). ترکیب درست: صفحه با `grid`، داخل هر ناحیه `flex`.

عمیق تر: `flex` محتوا-محور است (اندازه از محتوا می آید)، `grid` طرح-محور (شبکه تعریف می کنی). سوال دنباله: `justify-content` روی محور اصلی، `align-items` روی محور عرضی – و با `flex-direction` جایشان عوض می شود؛ `flex: 1` یعنی `grow:1 shrink:1 basis:0`.

Q13: یک باکس را دقیقاً وسط والدش چطور می گذاری؟ (سه روش)

جواب:

```
/* ۱) flex */      .parent { display: flex; justify-content: center; align-items: center; }
/* ۲) grid - کوتاه ترین */ .parent { display: grid; place-items: center; }
/* ۳) کلاسیک */      .child { position: absolute; top: 50%; left: 50%; transform: translate(-50%
```

عمیق تر: روش `absolute` برای مرکز کردن روی یک المان دیگر (tooltip) لازم است – و استفاده از `translate` به جای `top/left` یعنی بدون reflow (سوال Q18).

Q14: z-index چرا کار نمی کند؟ (stacking context)

جواب: `z-index` فقط در همان **stacking context** معنا دارد. هر المانی با `position non-static + z-index`، یا `opacity`، `transform`، `filter`، `< 1` و... یک `context` جدید می سازد – `z-index` فرزندان فقط داخل همان محدوده مقایسه می شود.

عمیق تر: باگ کلاسیک: مودال با `z-index ۹۹۹۸` داخل والدی با `transform` – زیر مودالی می ماند که بیرون آن با `z-index ۱۰` است. راه حل: `context` ها را تخت نگه دار و مقیاس سراسری `z-index` (مثلاً 100/200/300) تعریف کن. همین جا BFC را هم بگو: `overflow: hidden` یک BFC می سازد که `float` را محاصره و `margin-collapse` را قطع می کند – ابزار کلاسیک `clearing`.

Q15: چطور عنصری را مخفی کنم؟ (سه راه + sr-only)

جواب: `display: none` – از layout و درخت دسترسی‌پذیری حذف؛ `visibility: hidden` – جایش را نگه می‌دارد ولی نامرئی و غیرقابل تعامل؛ `opacity: 0` – نامرئی ولی قابل تعامل و **focus**! برای «فقط screen reader»: کلاس `sr-only` (absolute, 1px, clip).

عمیق‌تر: تله: `opacity: 0` روی input یعنی کاربر با tab می‌رسد و «چیزی که نیست» را تعامل می‌کند. اگر transition می‌خواهی، `visibility` را با تأخیر داخل transition ترکیب کن.

Q16: فرق pseudo-class و pseudo-element؟

جواب: pseudo-class وضعیت المان را هدف می‌گیرد (`:nth-child()` ، `:focus-visible` ، `:hover`)؛ pseudo-element بخشی از المان را می‌سازد (`::selection` ، `::placeholder` ، `::after` ، `::before`) – با خاصیت `content`.

عمیق‌تر: `::before/::after` برای دکور بدون DOM اضافه (خط، badge، آیکون)؛ `:focus-visible` به جای `:focus` تا کلیک موس outline نگیرد ولی کیبورد بگیرد. فرم صحیح pseudo-element های جدید: دو نقطه (`::`).

Q17: mobile-first یعنی چه؟ responsive و adaptive چه فرقی دارند؟

جواب: mobile-first یعنی CSS پایه برای موبایل نوشته شود و با `min-width` به سمت صفحه‌های بزرگ‌تر توسعه یابد. responsive: یک layout که با viewport انعطاف می‌گیرد؛ adaptive: نسخه‌های ثابت جدا برای هر breakpoint (مرسوم نیست).

عمیق‌تر: سوال چرخشی هندبوک: «@media غیر از screen؟» → `@media print` (استایل چاپ رزومه/فاکتور). و در grid های مدرن، `repeat(auto-fit, minmax(240px, 1fr))` جای چندین media query را می‌گیرد.

Q18: چرا برای انیمیشن از translate() به جای top/left؟

جواب: `top/left` تغییر layout می‌دهد (reflow + paint)؛ `transform` بعد از layout عمل می‌کند و فقط paint/composite می‌گیرد (GPU) – انیمیشن‌های 60fps با `transform` و `opacity` ساخته می‌شوند، نه خواص layout.

عمیق‌تر: این پل به فصل بعد است: reflow گران‌ترین مرحله pipeline رندر است (فصل ۱۵ Q6). ابزار دیدن: DevTools → Rendering → Paint flashing.

Q19: تم تاریک را با CSS چطور می‌سازی؟

جواب: متغیرهای CSS روی `:root` تعریف و زیر `[data-theme="dark"]` بازتعریف می‌شوند؛ کامپوننت‌ها فقط `var(--color-bg)` می‌خوانند؛ سوییچ‌کننده attribute را عوض می‌کند (با `next-themes` و `attribute="data-theme"`).

```
:root { --color-bg: #fff; --color-ink: #111; }
[data-theme="dark"] { --color-bg: #0f172a; --color-ink: #e2e8f0; }
.card { background: var(--color-bg); color: var(--color-ink); }
```

عمیق‌تر: (Sass) preprocessor روزی برای nesting و mixin بود؛ امروز nesting بومی CSS و متغیرها اکثر نیازها را پوشش می‌دهند. `color-scheme: light dark` را هم اعلام کن تا فرم‌ها و اسکریپت‌ها سیستم هماهنگ شوند.

Q20: فونت سفارشی چطور لود کنی که متن فلش نزنند؟

جواب: با `font-display: swap` (فوراً فونت جایگزین، بعد تعویض) یا `optional` (اگر نرسید اصلاً تعویض نکن)؛ با `preload` دانلود را زودتر شروع کن؛ فقط وزن‌های لازم را سابست کن.

عمیق‌تر: دو اصطلاح را بلد باش: **FOUT** (فلش متن بدون استایل) و **FOIT** (متن نامرئی تا رسیدن فونت – بدترین). در Next همه‌چیز حل‌شده است: `next/font` فونت را self-host و preload می‌کند و layout shift صفر می‌دهد (فصل ۷ Q16).

✓ جمع‌بندی فصل

- HTML: معنایی بنویس، `defer` بگذار، ابعاد تصویر را صریح بده، `lang/dir` را فراموش نکن
- CSS: border-box، specificity، سه‌رقمی، flex یک‌بعدی / grid دوطبقه‌ای، انیمیشن فقط transform/opacity
- تله‌های پرتکرار: z-index و sticky، stacking context، overflow: hidden، opacity: 0 و قابل‌تعامل، FOUT/FOIT فونت
- منبع این فصل: سوالات واقعی (h5bp) Front End Interview Handbook – بدون تکرار فصل‌های ۵-۷

📝 تمرین فصل ۱۴

1. Specificity را حدس بزن بدون اجرا: کدام می‌برد؟ `ul li a.active` / `.nav .link:hover` / `#nav .link` – بعد در DevTools (پنل Styles، strike-through شده‌ها) چک کن.
2. صفحه‌ای با یک باکس بساز و هر سه روش وسط‌چین (Q13) را عملی پیاده کن.
3. جدولی با هدر sticky بساز (Q11) – و بعد `overflow: hidden` روی والد بگذار تا شکستن sticky را ببینی.
4. کلاس `sr-only` را بدون کپی بنویس (Q15) – و با یک screen reader مرورگر تستش کن.
5. تم تاریک با `data-theme` + متغیرها بساز (Q19) – سه رنگ، دو تم.

▼ جواب تمرین ۱

`#nav .link` با (1,1,0) از همه بالاتر است (`ul li a.active` = (0,1,3)، `.nav .link:hover` = (0,3,0)). اگر دو سلکتور هم‌وزن بودند، ترتیب سورت‌بندی تصمیم می‌گرفت. **important**! از همه این‌ها بالاتر می‌پرید.

→ **فصل بعد:** ۱۵ سوال مرورگر، DOM و تله‌های JS – نیمه‌ی گم‌شده‌ی دیگر مصاحبه.

فصل ۱۵ – ۱۵ سوال مرورگر، DOM و تله‌های JS با جواب کامل

🎯 هدف: نیمه‌ی گمشده‌ی مصاحبه‌ها: DOM و مرورگر + سوالات «خروجی‌بگو». از سوالات واقعی **Front End Interview Handbook** و **JavaScript Questions** (۱۴۰+ سوال معروف Lydia Hallie) انتخاب شده – بدون تکرار فصل ۵.

💡 سوالات «خروجی این کد چیست؟» را بلند حل کن: اول حدس، بعد توضیح، بعد اجرا – همان‌طور که در مصاحبه باید حرف بزنی.

۱۵.۱ – DOM و مرورگر

Q1: سه فاز **event** چیست و **event delegation** چگونه کار می‌کند؟

جواب: رویداد سه فاز دارد: **capturing** (از window به پایین) → **target** (المان واقعی) → **bubbling** (به بالا).
delegation یعنی listener را روی والد گذاشتن و آیتم واقعی را از **event.target** پیدا کردن – یک listener به جای هزار تا، و روی آیتم‌های اضافه‌شده‌ی بعدی هم کار می‌کند:

```
list.addEventListener("click", (e) => {  
  const btn = e.target.closest("button.delete");  
  if (!btn) return; // کلیک روی جای دیگر نیست  
  removeItem(btn.dataset.id); // 🐱 delegation + data-* (Q5 فصل ۱۴)  
});
```

عمیق‌تر: دو ابزار متفاوت که همیشه با هم اشتباه گرفته می‌شوند: **stopPropagation** جلوی بالا رفتن رویداد؛ **preventDefault** جلوی رفتار پیش‌فرض مرورگر (لینک، submit، کلیک). و **event.target** = المان واقعی کلیک‌شده، **event.currentTarget** = المانی که listener رویش است. React هم ذاتاً **delegate** است (روی **root** – به همین دلیل React 17 به بالا attach به **root** است).

Q2: فرق **property** و **attribute** در DOM؟

جواب: **attribute** چیزی است که در HTML نوشته‌ای (`<input value="x">`)؛ **property** فیلد زنده‌ی آبجکت DOM (`input.value`). کاربر تایپ می‌کند → **property** عوض می‌شود، **attribute** همان اولی می‌ماند.
عمیق‌تر: `getAttribute("value")` مقدار اولیه HTML را می‌دهد؛ `input.value` مقدار فعلی. برای checkbox هم: **property** `checked` رفت‌وبرگشت دارد، **attribute** فقط `default` است. در React/JSX این مرز را فریم‌ورک مدیریت می‌کند – ولی سوال پشت سوال همین است.

Q3: فرق DOMContentLoaded و load؟

جواب: `DOMContentLoaded` : درخت DOM آماده است (منتظر تصویر/iframe نیست) – جای `init` های معمول؛ `load` : همه‌ی منابع هم رسیده‌اند – دیر. اسکریپت‌های `defer` (و ES Module ها) دقیقاً قبل از `DOMContentLoaded` اجرا می‌شوند (فصل ۱۴ Q3).

عمیق‌تر: در SPA معمولاً هیچ‌کدام را دستی نمی‌خواهی – `mount` شدن فریم‌ورک نقطه‌ی شروع توست؛ `load` برای کارهایی مثل خواندن ابعاد واقعی تصویر یا متریک‌ها.

Q4: فرق localStorage و cookie، sessionStorage؟

جواب: `cookie`: ~۴KB، با هر `request` به سرور می‌رود، `expiry` دارد و فلگ‌های امنیتی (`httpOnly/secure/SameSite`)؛ `localStorage`: ~۵-۱۰MB، دائمی، فقط کلاینت؛ `sessionStorage`: مثل `localStorage` ولی تا بستن تب.

عمیق‌تر: جواب تصمیمی بده: توکن → `cookie` با `httpOnly` (JS نمی‌خواندش – ضد XSS)؛ سید خرید/تنظیمات → UI `localStorage`؛ داده‌ی موقت تب → `sessionStorage`. نگهداشتن توکن در `localStorage` = سطح حمله‌ی XSS فعال (فصل ۷ Q15 امنیت Next).

Q5: Same-Origin Policy و CORS چیست؟

جواب: JS: SOP: یک `origin` فقط به `origin` خودش (پروتکل + دامنه + پورت) دسترسی دارد. **CORS مکانیزم استاندارد استثناست:** سرور با هدر `Access-Control-Allow-Origin` اعلام می‌کند کدام `origin` مجاز است.

عمیق‌تر: درخواست‌های «غیرساده» (POST با JSON، هدر سفارشی) اول **preflight** با متد `OPTIONS` می‌فرستند و سرور باید همان `route` را جوابش را بدهد. نکته‌ی کلیدی: کلاینت نمی‌تواند CORS را دور بزند – خطای CORS یعنی درد سرور است؛ راه‌حل فرانت: پروکسی سمت سرور (در Next: Route Handler یا `rewrites`)، نه تغییر `fetch`!

۱۵.۲ – تله‌های JS (فرمت «خروجی بگو»)

Q6: coercion – خروجی این‌ها؟

```
function sum(a, b) { return a + b; }
sum(1, "2");           // "12" - + چسباندن
3 + 4 + "5";           // "75" - بعد 7=(4+3)، 5"+7
"5" - 3;               // 2 - - 5؛ فقط ریاضی است؛ number
+true;                 // 1
[] + {};               // "[object Object]"
```

چرا: `+` دو کاره است – اگر یکی از عملوندها رشته باشد، چسباندن؛ `-` فقط عددی. تبدیل پیش‌فرض: `[] → ""` و `{}` `"[object Object]"` → (پایه‌ی `=` از فصل ۵ Q2).

typeof و instanceof – خروجی‌ها؟

```
typeof null;           // "object"   باگ تاریخی؛ سازگاری! هرگز فیکس نمی‌شود 🐞
typeof NaN;            // "number"
typeof typeof 1;       // "string"   - typeof 1 === "number", typeof آن "string"
new Number(3) === 3;    // false      - wrapper ≠ primitive
3 instanceof Number;   // false      - primitive معنا ندارد
```

عمیق‌تر: چک عدد واقعی: `Number.isNaN(x)` (نه `x === NaN`)، چک آرایه: `Array.isArray`، و برای همه چیز: `Object.prototype.toString.call(x)`.

Q8: falsy ها و ممیز شناور – خروجی؟

```
!!null; !!""; !!" "; !!0; !!NaN; !![]; !!"0";
// false false true false false true true
0.1 + 0.2 === 0.3; // false → 0.1 + 0.2 = 0.30000000000000004
```

چرا: شش `false, 0, "", null, undefined, NaN` – falsy: پس `" "` و `[]` و `"0"` truthy اند! ممیز شناور: مقایسه با آستانه – `Math.abs(a - b) < Number.EPSILON` – یا گرد کردن با `toFixed`.

Q9: کدام متدهای آرایه mutate می‌کنند؟ slice/splice

```
const a = [1, 2, 3, 4, 5];
a.slice(1, 3);           // [2, 3]    - سالم a کپی؛
const b = [1, 2, 3, 4, 5];
b.splice(1, 3);          // [2, 3, 4]  - حالا b [5, 1] برش زد؛
const c = [1];
c.push(6);               // 2         - طول جدید برمی‌گرداند، نه آرایه push!
[10, 1, 2].sort();       // [1, 10, 2] - مقایسه‌ی رشته‌ای!
```

چرا: mutating ها: `push/pop/shift/unshift/splice/sort/reverse/fill` – بقیه (`map/filter/slice/concat`) کپی می‌سازند. sort عددی: `(a, b) ≥ a - b`. در React هرگز mutate نکن – state جدید بساز (snapshot فصل ۶ Q5).

Q10: forEach یا map

جواب: `map` آرایه‌ی جدید با خروجی callback می‌سازد؛ `forEach` فقط پیمایش می‌کند و برنمی‌گرداند (`undefined`).

تله (دو خطای واقعی کد ریویو):

```
arr.map(x ⇒ console.log(x)); // [undefined, undefined, ...] - forEach می‌خواست!
const names = await Promise.all( // async فقط map داخل Promise می‌سازد
  ids.map(async (id) ⇒ getUser(id))
); // جمع کن Promise.all با 🐞
```

for...of و Q11: for...in

```
const arr = ["a", "b"];
for (const x in arr) console.log(x); // "0", "1" - کلیدها (index رشته)
for (const x of arr) console.log(x); // "a", "b" - مقادیر
```

چرا: `for...in` روی `enumerable keys` می‌چرخد (حتی میراث پروتوتایپ - فصل ۵ Q8)؛ `for...of` روی `iterables` (Array, String, Map, Set). برای آبجکت معمولی: `for...of` + `Object.entries(obj)` یا `Object.keys/values`.

Q12: this گمشده - خروجی؟

```
const circle = {
  radius: 10,
  diameter() { return this.radius * 2; },
  perimeter() => this.radius * 2 * Math.PI, // ➡ arrow آبجکت!
};
circle.diameter(); // 20
circle.perimeter(); // NaN - this می‌آید از enclosing آرو → undefined
const d = circle.diameter;
d(); // NaN (sloppy) یا TypeError (strict) - this از دست رفت
```

چرا: `this` arrow خودش ندارد (lexical - فصل ۵ Q6)؛ و متد جدا شده از آبجکت هم `this` را می‌بازد - راه‌حل: `bind` (فصل ۵ Q7) یا گرفتن متد داخل تابع دیگر.

Q13: زنجیره‌ی Promise - خروجی؟

```
Promise.resolve(1)
  .then(x => x + 1) // 2
  .then(x => { throw x; }) // catch پرش به نزدیک‌ترین
  .catch(() => 3) // زنجیره ادامه پیدا می‌کند - 3
  .then(x => x * 10) // 30
  .finally(() => console.log("done")) // "done" - ولی مقدار را عوض نمی‌کند
  .then(x => console.log(x)); // 30
```

چرا: هر `then` خروجی قبلی را می‌گیرد؛ `throw` تا نزدیک‌ترین `catch` می‌پرد؛ اگر مقدار برگرداند زنجیره «سالم» ادامه می‌یابد؛ `pass-through finally` است. (تکمیل `async/await` فصل ۵ Q10).

Q14: const و Object.freeze - این کد چه می‌کند؟

```
const user = { name: "M", tags: ["a"] };
user.name = "M2"; // فقط بازتخصیص را می‌بندد، نه تغییر را - مجاز ✓
Object.freeze(user);
user.name = "M3"; // بی‌صدا بی‌اثر (در strict: TypeError)
user.tags.push("b"); // ❌ freeze است! انجام می‌شود
```

چرا: `freeze` فقط `properties` سطح اول را قفل می‌کند؛ آبجکت‌های تودرتو هنوز مرجع مشترک‌اند (ریشه‌ی مشترک با `shallow/deep copy` فصل ۵ Q12). `deep freeze` دستی/کتابخانه‌ای است - و پاس مصاحبه: «برای `immutability` واقعی در `state`، خودم کپی می‌سازم».

Q15: ASI – چرا خروجی undefined است؟

```
function get() {  
  return  
  {  
    ok: true,  
  };  
}  
get(); // undefined – return ساختن نشده! همان‌جا تمام شد؛ آبجکت هرگز ساخته نشده!
```

چرا: Automatic Semicolon Insertion بعد از `return/throw/break/continue/new` پایان خط می‌گذارد؛ براکت خط بعد statement جدا و مرده است. قاعده: باز کردن پرانتز/براکت روی همان خط – به همین دلیل JSX چندخطی را می‌نویسی و Prettier هم همین را الزام می‌کند. (نکته strict: حالت strict بی‌صداها را ارور می‌کند – مثل تخصیص متغیر بدون تعریف.)

✓ جمع‌بندی فصل

- DOM: delegation = یک listener برای همه؛ `target ≠ currentTarget`؛ جلوگیری از انتشار (stopPropagation) ≠ جلوگیری از رفتار پیش‌فرض (preventDefault)
- مرورگر: توکن `httpOnly cookie` → خطای CORS = درد سرور؛ `DOMContentLoaded` جای `init`
- تله‌ها: `+` با رشته می‌چسباند، `sort`، `typeof null`، رشته‌ای، `forEach` بدون `freeze`، `return` سطحی، ASI بعد `return`
- منابع این فصل: سوالات واقعی هندبوک فرانت + کوئیز ۱۴۰ سوالی Lydia Hallie – خودت را با آن بزن: **روزی ۵ سوال**، **حدس بزن بعد اجرا**

📝 تمرین فصل ۱۵

۱. پنج سوال از **کوئیز JavaScript Questions** را انتخاب کن: بدون اجرا حدس بزن، بعد اجرا – و برای هر اشتباه، «چرا» را یک جمله بنویس.
۲. لیست TODO فصل ۸ را با delegation بازنویسی: یک listener روی `<ul>`، حذف/تیک با `*data-`.
۳. در پروژه‌ای که `fetch` بین‌دامنه‌ای دارد، یک بار عمداً CORS را بشکن و بعد با Route Handler پروکسی‌اش کن – تجربه‌ی واقعی برای جواب Q5.
۴. خروجی این را حدس بزن و اجرا کن:

```
const user = { name: "Ali", tags: ["x"] };  
Object.freeze(user);  
user.tags[0] = "y";  
console.log(user.tags[0]);
```

▼ جواب تمرین ۴

freeze – "y" فقط سطح اول را قفل می‌کند؛ آرایه‌ی داخل آبجکت هنوز mutable است. برای قفل کامل باید بازگشتی همه‌ی سطوح را freeze کنی (deep freeze).

➔ **بعدش کجاست؟** برگرد به **فصل ۱۳** – چیت‌شیت و چک‌لیست‌ها را با این ۳۵ سوال تازه تمرین کن، بعد سراغ مصاحبه‌های واقعی برو.